Mentry

Programmierhandbuch (deutsch)

2026-10-26

Inhalt

index **3**
mentry **3**
mentryDateTime **30**
mentryFixedPoint **35**
mentryIPAddr **37**
mentryIPv6Addr **38**
mentryThemes **41**
mentryWidget **44**
wcbRef **55**
Lizenz **66**

index

Stichwörter: index

Das Mehrfeld-Eingabe-Widget-Paket Mentry 4.5

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

Mentry-Programmierhandbuch
Der Befehl mentry::mentry
Mehrfeld-Eingabe-Widgets für Datum und Uhrzeit
Mehrfeld-Eingabe-Widgets für reelle Zahlen im Festkommaformat
Mehrfeld-Eingabe-Widgets für IP-Adressen
Mehrfeld-Eingabe-Widgets für IPv6-Adressen
Befehle zu Tile-Themen
Wcb-Befehlsreferenz

mentry

Stichwörter: mentry

Mentry-Programmierhandbuch

Für Mentry Version 4.5.1

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

Überblick
- Was ist Mentry?
- Woher bekomme ich es?
- Wie installiere ich es?
- Wie verwende ich es?
- Mehr zu Mentry_tile

Beispiele
- Ein Mentry-Widget für Telefonnummern
- Ein Mentry-Widget für Ethernet-Adressen

- Ein Mentry-Widget für Franking-IDs
- Verwendung von Mentry-Widgets für Datum und Uhrzeit
- Auf Tile aufbauende Demo-Skripte

Überblick

Was ist Mentry?

Mentry ist ein Paket für Tcl/Tk ab Version 8.4, das in reinem Tcl/Tk-Code geschrieben ist. Es enthält:

- die Implementierung des Mehrfeld-Eingabe-Megawidgets mentry, einschließlich eines allgemeinen Dienstprogrammmoduls für Megawidgets;
- Prozeduren, die die Verwendung von Mehrfeld-Eingabe-Widgets zur Anzeige und Bearbeitung von Datum und Uhrzeit in einer Vielzahl von Formaten, reellen Zahlen im Festkommaformat, sowie IP(v4)-Adressen und IPv6-Adressen;
- drei ausführlich kommentierte Demo-Skripte, die die typischen Schritte enthalten, die zur Erstellung und Handhabung eines Mentry-Widgets für einen bestimmten Zweck erforderlich sind;
- zwei Beispielskripte, die die Verwendung der Prozeduren für Datum und Uhrzeit veranschaulichen;
- auf Tile aufbauende Entsprechungen der oben genannten Demo-Skripte;
- dieses Tutorial;
- Referenzseiten im HTML-Format.

Ein Mehrfeld-Eingabe-Widget besteht aus einer beliebigen Anzahl von Eingabe-Widgets, die durch Beschriftungen voneinander getrennt und in einem Rahmen eingebettet sind. Durch entsprechend gewählte Konfigurationsoptionen wirkt dieses Gebilde wie ein einziges Eingabefeld, das vorgefertigte Textabschnitte enthält, deren Positionen innerhalb des Widgets unveränderlich sind. Die anfängliche Breite einer Eingabekomponente bestimmt auch die maximale Anzahl an Zeichen, die in sie eingefügt werden können; wird diese Grenze in einer Eingabekomponente erreicht, die den Fokus besitzt, wird dieser automatisch auf die nächste aktivierte Eingabekomponente gesetzt. Die gleiche Aktion wird ausgelöst, wenn ein Zeichen eingegeben wird, das in der Beschriftung hinter der aktuellen Eingabekomponente enthalten ist, sofern diese nicht leer ist.

Innerhalb eines Mentry-Widgets wirken die Tasten `Left`, `Right`, `Home`, `End` und `BackSpace` über die Grenzen der Eingabekomponenten hinweg, während `Control-Left` und `Control-Right` dieselbe Rolle spielen wie `Tab` und `Shift-Tab` bei gewöhnlichen Einträgen.

Einige der oben genannten Funktionen werden mithilfe des Widget-Rückruffunktions-Pakets Wcb implementiert, das ebenfalls in reinem Tcl/Tk-Code geschrieben ist. **Das Mentry-Paket erfordert Version 3.1 oder höher von Wcb**, das unter folgender Adresse heruntergeladen werden kann:

https://www.nemethi.de

Es ist ganz einfach, ein Mehrfeld-Eingabe-Widget zu erstellen. Zum Beispiel der Befehl

```
mentry::mentry .me -body {3 - 3 - 4}
```

wird ein Mentry-Widget erstellt, das aus zwei Einträgen mit der Breite 3 und einem Eintrag mit

der Breite 4 besteht, die durch die Zeichenfolge "-" voneinander getrennt sind. Mit dem Befehl

```
foreach w [.me entries] {
    wcb::cbappend $w before insert wcb::checkStrForNum
}
```

So können Sie sicherstellen, dass in den drei Eingabefeldern nur Ziffern eingegeben werden können, und bieten damit eine komfortable und sichere Benutzeroberfläche für die Bearbeitung von 10-stelligen Telefonnummern.

Woher bekomme ich es?

Mentry kann unter derselben URL wie Wcb kostenlos heruntergeladen werden. Die Distributionsdatei lautet `mentry4.5.1.tar.gz` für UNIX und `mentry4_5_1.zip` für Windows. Diese Dateien enthalten dieselben Informationen, mit Ausnahme des zusätzlichen Wagenrücklaufzeichens, das in den Textdateien für Windows dem Zeilenvorschub am Ende jeder Zeile vorangestellt ist.

Mentry ist ebenfalls in tklib enthalten, dessen Adresse lautet

https://core.tcl.tk/tklib

Wie installiere ich es?

Installieren Sie das Paket als Unterverzeichnis eines der Verzeichnisse, die durch die Variable `auto_path` angegeben sind. Sie können es beispielsweise als Unterverzeichnis des Verzeichnisses `lib` innerhalb Ihrer Tcl/Tk-Installation installieren.

Um Mentry *unter UNIX* zu installieren, wechseln Sie mit `cd` in das gewünschte Verzeichnis und entpacken Sie die Distributionsdatei `mentry4.5.1.tar.gz`:

```
gunzip -c mentry4.5.1.tar.gz | tar -xf -
```

Auf den meisten UNIX-Systemen kann dies durch Folgendes ersetzt werden:

```
tar -zxf mentry4.5.1.tar.gz
```

Beide Befehle erstellen ein Verzeichnis namens `mentry4.5.1` mit den Unterverzeichnissen `demos`, `doc` und `scripts`.

Unter Windows verwenden Sie WinZip oder ein anderes Programm, mit dem Sie die Distributionsdatei `mentry4_5_1.zip` in das Verzeichnis `mentry4.5.1` entpacken können, einschließlich der Unterverzeichnisse `demos`, `doc` und `scripts`.

Die Datei `mentryThemes.tcl` im Verzeichnis `scripts` wird nur für Anwendungen benötigt, die das Paket "Mentry_tile" verwenden (siehe nächster Abschnitt).

Beachten Sie, dass in tklib das Mentry-Verzeichnis `demos` durch das Unterverzeichnis `mentry` des Verzeichnisses `examples` ersetzt wird. Bitte berücksichtigen Sie dies beim Lesen der folgenden Beispiele.

Wie verwende ich es?

Die Mentry-Distribution umfasst zwei Pakete: **Mentry** und **Mentry_tile**. Der Hauptunterschied zwischen den beiden besteht darin, dass Mentry_tile die auf Tile aufbauende, themenspezifische Darstellung von Mentry-Widgets ermöglicht; dieses Paket erfordert Tile 0.6 oder höher. Es ist nicht möglich, beide Pakete in ein und derselben Anwendung zu verwenden, da beide im selben

Namespace `mentry` implementiert sind und identische Befehle bereitstellen.

Um auf die Befehle und Variablen des Mentry-Pakets zugreifen zu können, müssen Ihre Skripte eine der folgenden Zeilen enthalten:

```
package require mentry ?version?
package require Mentry ?version?
```

Sie können eine der beiden oben genannten Anweisungen verwenden, da die Datei `mentry.tcl` beide Zeilen enthält.

```
package provide mentry ...
package provide Mentry ...
```

Um auf die Befehle und Variablen des Pakets "Mentry_tile" zugreifen zu können, müssen Ihre Skripte ebenfalls eine der folgenden Zeilen enthalten:

```
package require mentry_tile ?version?
package require Mentry_tile ?version?
```

Auch hier können Sie eine der beiden oben genannten Anweisungen verwenden, da die Datei `mentry_tile.tcl` beide Zeilen enthält.

```
package provide mentry_tile ...
package provide Mentry_tile ...
```

Es steht Ihnen frei, jeweils eine der oben genannten Zeilen aus `mentry.tcl` bzw. `mentry_tile.tcl` zu entfernen, wenn Sie verhindern möchten, dass die entsprechenden Pakete jeweils unter zwei verschiedenen Beschreibungen angezeigt werden. Dadurch beschränken Sie natürlich das Argument von `package require` auf einen einzigen Namen pro Paket.

Da die Pakete "Mentry" und "Mentry_tile" im Namespace `mentry` implementiert sind, müssen Sie entweder die

```
namespace import mentry::pattern ?mentry::pattern ...?
```

Befehl zum Importieren der benötigten *Prozeduren* oder verwenden Sie qualifizierte Namen wie `mentry::mentry`. In den folgenden Beispielen haben wir uns für den letzteren Ansatz entschieden.

Um auf Mentry-*Variablen* zuzugreifen, *müssen* Sie qualifizierte Namen verwenden. Es gibt nur drei Mentry-Variablen (und eine weitere bei Verwendung von Mentry_tile), die für den Zugriff außerhalb des Namespaces `mentry` vorgesehen sind:

- Die Variable `mentry::version` enthält die aktuelle Versionsnummer des Mentry-Pakets.
- Die Variable `mentry::library` enthält den Pfad zum Mentry-Installationsverzeichnis.
- Die schreibgeschützte Variable `mentry::usingTile` hat im Paket "Mentry" den Wert `0` und in "Mentry_tile" den Wert `1`.
- In "Mentry_tile" enthält das Array `mentry::themeDefaults` die themenspezifischen Standardwerte einer Reihe von Mentry-Konfigurationsoptionen.

Mehr zu Mentry_tile

Wie oben erwähnt, besteht ein Mentry-Widget aus Entry- und Label-Widgets, die in einen Frame

eingebettet sind. Während im Mentry-Paket alle diese Komponenten Tk-Widgets sind, verwendet das Mentry_tile-Paket sowohl Tk-Frame- und Tile-Entry- als auch Tk-Label-Widgets. Aufgrund mehrerer Inkompatibilitäten zwischen Tk und Tile ist es derzeit nicht möglich, alle Tk-Widgets, aus denen ein Mentry besteht, durch ihre Tile-Entsprechungen zu ersetzen. Tatsächlich sind die "Entry"-Komponenten eines Tile-basierten Mentry in Tk-"Frame"-Widgets eingebettet, die wiederum zusammen mit den "Label"-Widgets in ein speziell konstruiertes "Tile-Entry"-Widget statt in einen "Frame" gepackt sind. Dieses etwas komplizierte Layout ist erforderlich, da es in einigen Themen nicht möglich ist, flache, randlose "Tile-Entry"-Widgets zu zeichnen.

Daraus folgt, dass **das Mentry_tile-Paket nur dann wie erwartet funktioniert, wenn die Tk-Befehle "frame" und "label" nicht durch den Befehl Namespace import -force ttk::* im globalen Geltungsbereich überschrieben wurden**. Während in früheren Tile-Versionen empfohlen wurde, diesen Befehl im globalen Geltungsbereich zu verwenden – für die wirklich Abenteuerlustigen –, gilt dies in neueren Tile-Versionen als wirklich schlechte Idee, die zu zahlreichen Fehlfunktionen führt. Stattdessen **sollten Sie explizit ttk::frame, ttk::label usw. aufrufen, wann immer Sie ein Tile-Widget verwenden möchten**.

Eine weitere Einschränkung, die zu berücksichtigen ist, ergibt sich daraus, dass in früheren Tile-Versionen der Befehl `(ttk::)style theme use` nur zum Festlegen des aktuellen Themas verwendet werden konnte, nicht jedoch zum Abrufen desselben. Aus diesem Grund greift das Mentry_tile-Paket, wenn es das aktuelle Thema nicht mithilfe von `ttk::style theme use` abrufen kann, auf die Variable `ttk::currentTheme` oder `tile::currentTheme` (je nach Tile-Version) zurück, die durch den Befehl `ttk::setTheme` oder `tile::setTheme` gesetzt wird. Daraus folgt, dass**, wenn die verwendete Tile-Version den Befehl ttk::style ohne Argument nicht unterstützt, die auf Tile aufbauenden Mentry-Widgets nur dann das erwartete Erscheinungsbild aufweisen, wenn das plattformspezifische Standardthema entweder unverändert bleibt oder durch Aufruf der Prozedur der Bibliothek ttk::setTheme oder tile::setTheme – je nach Tile-Version – durch ein anderes Thema ersetzt wird**. (Siehe auch den Befehl `mentry::setTheme`.)

Nach diesen Hinweisen zur Verwendung von "tile" werden im weiteren Verlauf dieses Abschnitts die Unterschiede zwischen den Paketen "Mentry" und "Mentry_tile" beschrieben.

Das Paket "Mentry_tile" überprüft, ob die erforderlichen Tk- und Tile-Versionen vorhanden sind, indem es die folgenden Befehle ausführt:

```
package require Tk 8.4-
if {$::tk_version < 8.5 || [regexp {^8\.5a[1-5]$} $::tk_patchLevel]} {
    package require tile 0.6-
}
```

Der zweite Befehl oben spiegelt die Tatsache wider, dass "Tile" ab Tk 8.5a6 in den Tk-Kern integriert ist und daher nur bei Verwendung einer früheren Tk-Version explizit geladen werden sollte.

Abgesehen davon und vom Suffix `_tile` im Befehl `package require` besteht der einzige Unterschied (aus Sicht des Programmierers) zwischen den Paketen "Mentry" und "Mentry_tile" in den unterstützten Konfigurationsoptionen: Die folgenden Tk-(Entry-)Widget-Optionen, die im Mentry-Paket vorhanden sind, werden von Mentry_tile nicht unterstützt, da sie für Tile-(Entry-)Widgets nicht verfügbar sind: `-borderwidth`, `-disabledbackground`, `-disabledforeground`, `-highlightbackground`, `-highlightcolor`,

`-highlightthickness`, `-insertbackground`, `-insertborderwidth`, `-insertofftime`, `-insertontime`, `-insertwidth`, `-readonlybackground`, `-relief`, `-selectbackground`, `-selectborderwidth` und `-selectforeground`.

Beachten Sie, dass die Option `-background` nicht wie erwartet funktioniert, wenn das aktuelle Thema `Arc`, `plastik`, `tileqt`, `vista` oder `xpnative` ist, da diese Themen jeden Versuch, die Farbe des Hintergrunds eines Tile-entry-Widgets zu ändern, stillschweigend ignorieren.

Beachten Sie abschließend, dass wenn das Thema `tileqt` verwendet wird, die Versionsnummer des Pakets `tile::theme::tileqt` mindestens 0.4 betragen muss, und dass `tileqt` selbst nicht mit Tile-Versionen vor 0.7 kompatibel ist.

Beispiele

Ein Mentry-Widget für Telefonnummern

Lassen Sie uns das im Überblick erwähnte Beispiel etwas systematischer aufgreifen. Zunächst schreiben wir eine Prozedur zum Erstellen eines "mentry"-Widgets, mit dem sich 10-stellige Telefonnummern anzeigen und bearbeiten lassen und das alle vom Befehl `mentry::mentry` unterstützten Konfigurationsoptionen akzeptiert:

```
#-------------------------------------------------------------------------
    --------
# phoneNumberMentry
#
# Creates a new mentry widget win that allows to display and edit
    10-digit
# phone numbers.  Sets the type attribute of the widget to PhoneNumber
    and
# returns the name of the newly created widget.
#-------------------------------------------------------------------------
    --------
proc phoneNumberMentry {win args} {
    #
    # Create a mentry widget consisting of two entries of width 3 and
    one of
    # width 4, separated by "-" characters, and set its type to
    PhoneNumber
    #
    eval [list mentry::mentry $win] $args
    $win configure -body {3 - 3 - 4}
    $win attrib type PhoneNumber

    #
    # Allow only decimal digits in all entry components; use
    # wcb::cbappend (or wcb::cbprepend) instead of wcb::callback
    # in order to keep the wcb::checkEntryLen callback,
    # registered by mentry::mentry for all entry components
    #
```

```
    for {set n 0} {$n < 3} {incr n} {
        set w [$win entrypath $n]
        wcb::cbappend $w before insert wcb::checkStrForNum
        $win adjustentry $n "0123456789"
        bindtags $w [linsert [bindtags $w] 1 MentryPhoneNumber]
    }

    return $win
}
```

Das erste Argument `win` ist der Name des Widgets, und das Schlüsselwort `args` steht für eine Liste von Konfigurationsoptionen und deren Werten, genau wie bei den Standard-Tk-Widgets. Der Wert `{3 - 3 - 4}` der Option `-body` legt fest, dass der mentry aus zwei Einträgen mit der Breite 3 und einem mit der Breite 4 bestehen soll, getrennt durch Beschriftungen, die das Zeichen `"-"` anzeigen.

Jedes Mentry-Widget kann eine beliebige Anzahl privater **Attribute** besitzen, die mithilfe des Unterbefehls `attrib` der dem Widget entsprechenden Tcl-Prozedur gesetzt und abgerufen werden können. Wir verwenden diesen Unterbefehl, um das Attribut `type` des neu erstellten Widgets zu definieren und es auf den Wert `"PhoneNumber"` zu setzen. Dies ist zwar nicht unbedingt erforderlich, ermöglicht es uns jedoch, ein "mentry"-Widget für Telefonnummern von anderen Mehrfeld-Eingabe-Widgets zu unterscheiden.

Der Befehl `mentry::mentry` registriert die Rückruffunktion `wcb::checkEntryLen` für jede Eingabekomponente des mentry-Widgets, um die Anzahl der Zeichen, die dort eingegeben werden können, auf die in der Option `-body` festgelegte Anfangsbreite zu beschränken. Neben dieser Einschränkung sollen unsere Eingabefelder nur Dezimalziffern akzeptieren; daher verwenden wir den Befehl `wcb::cbappend`, um die Prozedur `wcb::checkStrForNum` zur Rückruffunktion jeder Eingabekomponente *hinzuzufügen*. Durch den Aufruf von `wcb::callback` anstelle von `wcb::cbappend` (oder `wcb::cbprepend`) würden wir die Liste der Rückruffunktionen durch diejenige *ersetzen*, die aus dem einzigen Element `wcb::checkStrForNum` besteht.

Nun wissen wir, dass jede Eingabekomponente des "mentry"-Widgets nur eine begrenzte Anzahl von Dezimalstellen akzeptiert. Aber sind die Breiten der Eingabefelder groß genug, um die maximale Anzahl von jeweils 3 bzw. 4 Dezimalstellen aufzunehmen? Bei einer Schriftart mit fester Zeichenbreite lautet die Antwort eindeutig "Ja", und dasselbe gilt für die meisten proportionalen Schriftarten. Es gibt jedoch Schriftarten, bei denen nicht alle Dezimalziffern die gleiche Breite haben. Aus diesem Grund rufen wir für jede Eingabekomponente den Unterbefehl `adjustentry` auf und übergeben ihm als letztes Argument eine Zeichenkette, die aus den zulässigen Zeichen besteht – in diesem Beispiel sind das die Dezimalziffern. Dieser Unterbefehl vergrößert bei Bedarf die Breite des Eingabe-Widgets, sodass es gerade groß genug für Texte der komponenten-spezifischen maximalen Länge ist, die aus den durch diese Zeichenkette festgelegten Zeichen bestehen.

Unsere zweite Prozedur gibt eine Telefonnummer an ein Mentry-Widget aus, dessen Attributwert `type` den Wert `"PhoneNumber"` hat:

```
#-------------------------------------------------------------------------
    --------
# putPhoneNumber
#
# Outputs the phone number num to the mentry widget win of type
    PhoneNumber.
# The phone number must be a string of length 10, consisting of
    decimal digits.
#-------------------------------------------------------------------------
    --------
proc putPhoneNumber {num win} {
    #
    # Check the syntax of num
    #
    if {[string length $num] != 10 || ![regexp {^[0-9]*$} $num]} {
        return -code error "expected 10 decimal digits but got
    \"$num\""
    }

    #
    # Check the widget and display the properly formatted phone number
    #
    checkIfPhoneNumberMentry $win
    $win put 0 [string range $num 0 2] [string range $num 3 5] \
               [string range $num 6 9]
}
```

Wir verwenden den Unterbefehl `put` der Tcl-Prozedur, die dem "mentry"-Widget entspricht, um die drei Teilzeichenfolgen der angegebenen Telefonnummer in den entsprechenden Einträgen anzuzeigen, beginnend mit dem Eintrag, dessen Index als erstes Argument nach dem Wort `put` festgelegt ist.

Als Nächstes benötigen wir eine Prozedur, die die Telefonnummer gibt, die in einem "mentry"-Widget enthalten ist, dessen Attributwert `type` den Wert `"PhoneNumber"` hat:

```
#-------------------------------------------------------------------------
    --------
# getPhoneNumber
#
# Returns the phone number contained in the mentry widget win of type
# PhoneNumber.
#-------------------------------------------------------------------------
    --------
proc getPhoneNumber win {
    #
    # Check the widget
    #
```

```
    checkIfPhoneNumberMentry $win

    #
    # Generate an error if any entry component is empty or incomplete
    #
    for {set n 0} {$n < 3} {incr n} {
        if {[$win isempty $n]} {
            focus [$win entrypath $n]
            return -code error EMPTY
        }
        if {![$win isfull $n]} {
            focus [$win entrypath $n]
            return -code error INCOMPL
        }
    }

    #
    # Return the phone number built from the
    # values contained in the entry components
    #
    $win getarray strs
    return $strs(0)$strs(1)$strs(2)
}
```

Die Prozedur durchläuft die INDIZES der Eintragskomponenten des angegebenen "mentry"-Widgets und ruft die Unterbefehle `isempty` und `isfull` des Tcl-Befehls auf, der dem angegebenen "mentry"-Widget entspricht. Wenn festgestellt wird, dass einer der Einträge leer oder unvollständig ist, ermittelt die Prozedur dessen Pfadnamen durch Aufruf des Unterbefehls `entrypath`, setzt den Fokus auf diesen Eintrag, löst einen Fehler aus und gibt den Wert `"EMPTY"` bzw. `"INCOMPL"` zurück. Die Anwendung, die diese Prozedur aufruft, sollte dann eine dem Wert der Rückgabe entsprechende Fehlermeldung anzeigen.

Beachten Sie, dass die Nummer `3` in der obigen Schleife `for` nichts anderes ist als `[$win entrycount]`. Außerdem würde es ausreichen, zu prüfen, ob alle Komponenten eines Eintrags vollständig sind, da ein leerer Eintrag gleichzeitig unvollständig ist. Die vorläufige Prüfung, ob ein Eintrag leer ist, dient lediglich der Benutzerfreundlichkeit.

Um aus den in den Eingabekomponenten enthaltenen Werten die Telefonnummer zu bilden, verwenden wir eine temporäre Array-Variable und rufen den Unterbefehl `getarray` auf, der den Inhalt der Einträge in die entsprechenden Array-Elemente kopiert.

Die beiden zuletzt vorgestellten Prozeduren enthalten einen Aufruf des Befehls `checkIfPhoneNumberMentry`, der wie folgt implementiert ist:

```
#-------------------------------------------------------------------------
    --------
# checkIfPhoneNumberMentry
#
# Generates an error if win is not a mentry widget of type PhoneNumber.
```

```
#-------------------------------------------------------------------------
    --------
proc checkIfPhoneNumberMentry win {
    if {![winfo exists $win]} {
        return -code error "bad window path name \"$win\""
    }

    if {[winfo class $win] ne "Mentry" ||
        [$win attrib type] ne "PhoneNumber"} {
        return -code error \
               "window \"$win\" is not a mentry widget for phone
    numbers"
    }
}
```

Diese Prozedur ruft den Wert des Attributs `type` ihres Arguments ab, um zu prüfen, ob es sich bei diesem um ein Mentry-Widget für Telefonnummern handelt (beachten Sie, dass dieses Attribut in der Prozedur `phoneNumberMentry` auf den Wert `"PhoneNumber"` gesetzt wurde).

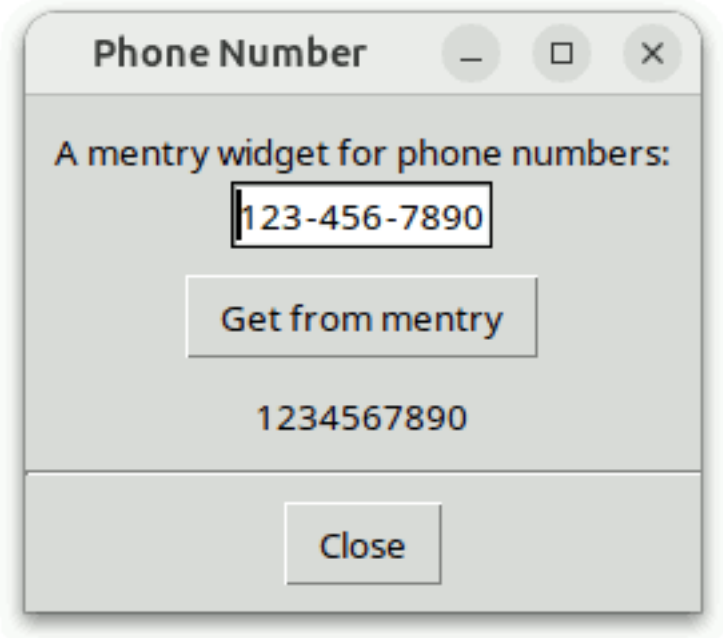
Denken Sie abschließend daran, dass wir in der Prozedur `phoneNumberMentry` das Bindungs-Tag `MentryPhoneNumber` in die Liste der Bindungs-Tags der Eintragskomponenten des Mentrys eingefügt haben, und zwar direkt hinter deren Pfadnamen. Das im folgenden Code gezeigte Skript für die Bindung dieses Tags verarbeitet die Ereignisse `<<Paste>>` (`<Control-v>` oder `<Command-v>`) bei diesen Widgets, indem er den aktuellen Inhalt der Zwischenablage in den Mentry einfügt, vorausgesetzt, es handelt sich um eine gültige 10-stellige Telefonnummer:

```
bind MentryPhoneNumber <<Paste>> { pastePhoneNumber %W }

#-------------------------------------------------------------------------
    --------
# pastePhoneNumber
#
# Handles <<Paste>> events in the entry component w of a mentry widget
    for
# 10-digit phone numbers by pasting the current contents of the
    clipboard into
# the mentry if it is a valid 10-digit phone number.
#-------------------------------------------------------------------------
    --------
proc pastePhoneNumber w {
    set res [catch {::tk::GetSelection $w CLIPBOARD} num]
    if {$res == 0} {
        set win [winfo parent $w]
        catch { putPhoneNumber $num $win }
    }

    return -code break ""
}
```

Die fünf oben beschriebenen Prozeduren sind in der Datei `phonenumber.tcl` implementiert, die sich im Verzeichnis `demos` befindet. Dieses Skript fasst sie zudem zu einer kleinen Anwendung zusammen, die die folgende Abbildung anzeigt:



Telefonnummer

Hier der maßgebliche Ausschnitt:

```
package require mentry

set title "Phone Number"
wm title . $title

#
# Add some entries to the Tk option database
#
source [file join [file dirname [info script]] option.tcl]

. . .

#
# Frame .f with a mentry displaying a phone number
#
frame .f
label .f.l -text "A mentry widget for phone numbers:"
phoneNumberMentry .f.me
pack .f.l .f.me

#
# Message strings corresponding to the values
# returned by getPhoneNumber on failure
#
array set msgs {
    EMPTY       "Field value missing"
    INCOMPL     "Incomplete field value"
}

#
# Button .get invoking the procedure getPhoneNumber
#
button .get -text "Get from mentry" -command {
    if {[catch {
        set num ""
        set num [getPhoneNumber .f.me]
    } result] != 0} {
        bell
        tk_messageBox -icon error -message $msgs($result) \
                      -title $title -type ok
    }
}

#
# Label .num displaying the result of getPhoneNumber
#
```

```
label .num -textvariable num

. . .

putPhoneNumber 1234567890 .f.me
focus [.f.me entrypath 0]
```

Ein Mentry-Widget für Ethernet-Adressen

Ethernet-Adressen werden in der Regel in der Form `"XX:XX:XX:XX:XX:XX"` geschrieben, wobei jedes `"X"` eine hexadezimale Ziffer ist. Die Datei `ethernetaddr.tcl` im Verzeichnis `demos` enthält die erforderlichen Schritte zum Erstellen und Verwenden eines Mehrfeld-Eingabe-Widgets zur Anzeige und Bearbeitung von Ethernet-Adressen. Sie implementiert die Prozeduren `ethernetAddrMentry`, `putEthernetAddr` und `getEthernetAddr`; wobei die beiden letzten die Prozedur `checkIfEthernetAddrMentry` aufrufen, während die erste wie folgt implementiert ist:

```
#--------------------------------------------------------------------
    --------
# ethernetAddrMentry
#
# Creates a new mentry widget win that allows to display and edit
    Ethernet
# addresses.  Sets the type attribute of the widget to EthernetAddr
    and returns
# the name of the newly created widget.
#--------------------------------------------------------------------
    --------
proc ethernetAddrMentry {win args} {
    #
    # Create a mentry widget consisting of 6 entries of width
    # 2, separated by colons, and set its type to EthernetAddr
    #
    eval [list mentry::mentry $win] $args
    $win configure -body {2 : 2 : 2 : 2 : 2 : 2}
    $win attrib type EthernetAddr

    #
    # Install automatic uppercase conversion and allow only hexadecimal
    # digits in all entry components; use wcb::cbappend (or
    wcb::cbprepend)
    # instead of wcb::callback in order to keep the wcb::checkEntryLen
    # callback, registered by mentry::mentry for all entry components
    #
    for {set n 0} {$n < 6} {incr n} {
        set w [$win entrypath $n]
        wcb::cbappend $w before insert wcb::convStrToUpper \
```

```
                        {wcb::checkStrForRegExp {^[0-9A-F]*$}}
        $win adjustentry $n "0123456789ABCDEF"
        bindtags $w [linsert [bindtags $w] 1 MentryEthernetAddr]
    }

    return $win
}
```

Beachten Sie erneut den Aufruf des Unterbefehls `adjustentry` des Tcl-Befehls, der dem Mentry-Widget zugeordnet ist, für jede seiner Eingabekomponenten. Dies ist notwendig, da bei einer Schriftart mit proportionalem Zeichenabstand die Zeichen `A` – `F` mehr Platz benötigen als die Ziffern `0` – `9` (und es ist nicht einmal garantiert, dass letztere die gleiche Breite haben).

Die Prozedur `putEthernetAddr` erwartet als erstes Argument eine Zeichenkette der Form `"XX:XX:XX:XX:XX:XX"`, wobei jedes `"XX"` eine hexadezimale Zeichenkette im Bereich von `0` bis `255` sein muss:

```
#--------------------------------------------------------------------
    --------
# putEthernetAddr
#
# Outputs the Ethernet address addr to the mentry widget win of type
# EthernetAddr.  The address must be a string of the form
    XX:XX:XX:XX:XX:XX,
# where each XX must be a hexadecimal string in the range 0 - 255.
    Leading
# zeros are allowed (but not required), hence the components may have
    more (but
# also less) than two characters; the procedure displays them with
    exactly two
# digits.
#--------------------------------------------------------------------
    --------
proc putEthernetAddr {addr win} {
    set errorMsg "expected an Ethernet address but got \"$addr\""

    #
    # Check the syntax of addr
    #
    set lst [split $addr :]
    if {[llength $lst] != 6} {
        return -code error $errorMsg
    }

    #
    # Try to convert the 6 components of addr to hexadecimal
    # strings and check whether they are in the range 0 - 255
```

```
    #
    for {set n 0} {$n < 6} {incr n} {
        set val 0x[lindex $lst $n]
        if {[catch {format "%02X" $val} str$n] != 0 || $val < 0 ||
    $val > 255} {
            return -code error $errorMsg
        }
    }

    #
    # Check the widget and display the properly formatted Ethernet
    address
    #
    checkIfEthernetAddrMentry $win
    $win put 0 $str0 $str1 $str2 $str3 $str4 $str5
}
```

Die Prozedur `getEthernetAddr` löst einen Fehler aus, wenn eine der Komponenten des angegebenen Mentry-Widgets leer ist. Sie akzeptiert zwar auch Eingabezeichenfolgen der Länge eins, im Rückgabewert haben jedoch alle Komponenten genau zwei Ziffern:

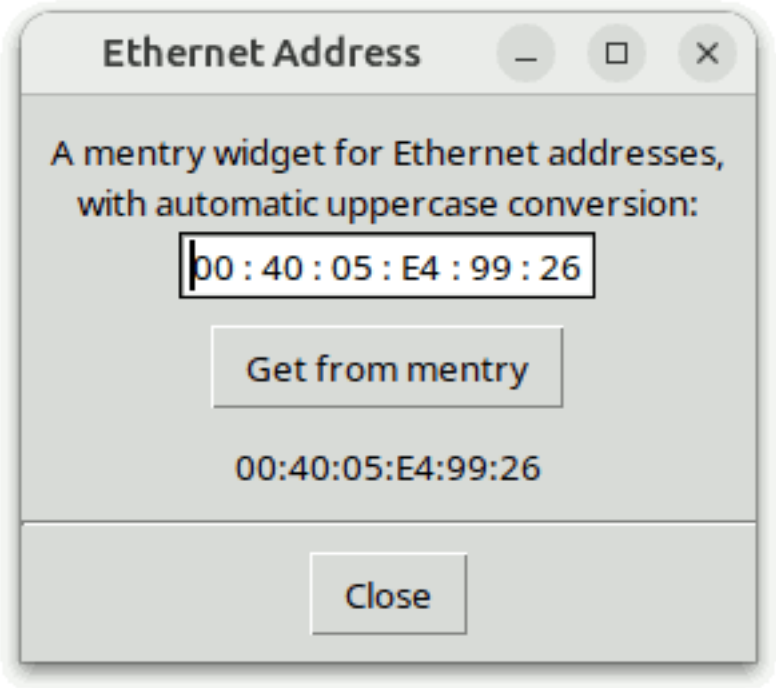
```
#-------------------------------------------------------------------
    --------
# getEthernetAddr
#
# Returns the Ethernet address contained in the mentry widget win of
    type
# EthernetAddr.
#-------------------------------------------------------------------
    --------
proc getEthernetAddr win {
    #
    # Check the widget
    #
    checkIfEthernetAddrMentry $win

    #
    # Generate an error if any entry component is empty
    #
    for {set n 0} {$n < 6} {incr n} {
        if {[$win isempty $n]} {
            focus [$win entrypath $n]
            return -code error EMPTY
        }
    }

    #
```

```
    # Return the properly formatted Ethernet address built
    # from the values contained in the entry components
    #
    $win getarray strs
    return [format "%02X:%02X:%02X:%02X:%02X:%02X" \
            0x$strs(0) 0x$strs(1) 0x$strs(2) 0x$strs(3) 0x$strs(4)
    0x$strs(5)]
}
```

Den restlichen Code werden wir hier nicht zeigen, da er dem im vorangegangenen Abschnitt vorgestellten Code sehr ähnlich ist. Das Mentry-Widget für Ethernet-Adressen sieht wie im folgenden Screenshot aus:



Ethernet-Adresse

Ein Mentry-Widget für Franking-IDs

Die Deutsche Post verwendet 20-stellige Frankier-IDs zur Nachverfolgung und Sortierung von Postsendungen. Frankier-IDs werden in der Regel als Zeichenfolgen der Form `"XX XXXX XXXX XX XXXX XXXX"` geschrieben, wobei jedes `"X"` eine hexadezimale Ziffer in Großbuchstaben ist und die letzte Ziffer eine CRC-Prüfsumme vom Typ CCITT (CRC-4) darstellt. Die Datei `frankingid.tcl` im Verzeichnis `demos` enthält die erforderlichen Schritte zum Erstellen und Verwenden eines Mehrfeld-Eingabe-Widgets zur Anzeige und Bearbeitung von Frankier-IDs. Sie implementiert die Hauptprozeduren `frankingIdMentry`, `putFrankingId` und `getFrankingId` sowie die Hilfsprozeduren `checkIfFrankingIdMentry`, `crc4` und `pasteFrankingId`.

Die Prozedur `frankingIdMentry`, die ein Mentry-Widget für Frankier-IDs anlegt, ist wie folgt implementiert:

```
#-------------------------------------------------------------------------
    --------
# frankingIdMentry
#
# Creates a new mentry widget win that allows to display and edit
    20-character
# Franking IDs.  Sets the type attribute of the widget to FrankingId
    and
# returns the name of the newly created widget.
#-------------------------------------------------------------------------
    --------
proc frankingIdMentry {win args} {
    #
    # Create a mentry widget consisting of 6 entries of widths 2, 4,
    4, 2, 4,
    # and 4, separated by " " characters, and set its type to
    FrankingId
    #
    eval [list mentry::mentry $win] $args
    $win configure -body {2 " " 4 " " 4 " " 2 " " 4 " " 4}
    $win attrib type FrankingId

    #
    # Install automatic uppercase conversion and allow only hexadecimal
    # digits in all entry components; use wcb::cbappend (or
    wcb::cbprepend)
    # instead of wcb::callback in order to keep the wcb::checkEntryLen
    # callback, registered by mentry::mentry for all entry components
    #
    for {set n 0} {$n < 6} {incr n} {
        set w [$win entrypath $n]
        wcb::cbappend $w before insert wcb::convStrToUpper \
                      {wcb::checkStrForRegExp {^[0-9A-F]*$}}
        $win adjustentry $n "0123456789ABCDEF"
        bindtags $w [linsert [bindtags $w] 1 MentryFrankingId]
    }

    return $win
}
```

Die Prozedur `putFrankingId` akzeptiert als erstes Argument sowohl eine unformatierte (20 Zeichen lange) Frankier-ID als auch eine im Format `"XX XXXX XXXX XX XXXX XXXX"`:

```
#-------------------------------------------------------------------------
    --------
# putFrankingId
#
```

```
# Outputs the Franking ID id to the mentry widget win of type
    FrankingId.  The
# Franking ID must be either a string consisting of 20 uppercase
    hexadecimal
# digits, or a string of the form "XX XXXX XXXX XX XXXX XXXX", where
    each X is
# an uppercase hexadecimal digit.
#-------------------------------------------------------------------
    --------
proc putFrankingId {id win} {
    #
    # Check the syntax of num
    #
    set expr1 {^[0-9A-F]{20}$}
    set expr2 {^[0-9A-F]{2} [0-9A-F]{4} [0-9A-F]{4} [0-9A-F]{2}
    [0-9A-F]{4}\
                [0-9A-F]{4}$}
    if {[regexp $expr1 $id]} {
        set isFormatted 0
    } elseif {[regexp $expr2 $id]} {
        set isFormatted 1
    } else {
        return -code error "expected a Franking ID but got \"$id\""
    }

    #
    # Check the widget and display the properly formatted Franking ID
    #
    checkIfFrankingIdMentry $win
    if {$isFormatted} {
        foreach {str0 str1 str2 str3 str4 str5} [split $id " "] {}
        $win put 0 $str0 $str1 $str2 $str3 $str4 $str5
    } else {
        $win put 0 [string range $id 0 1] [string range $id 2 5] \
                   [string range $id 6 9] [string range $id 10 11] \
                   [string range $id 12 15] [string range $id 16 19]
    }
}
```

Ebenso kann die von der Prozedur `getFrankingId` zurückgegebene Frankierungs-ID je nach dem booleschen Wert ihres optionalen Arguments entweder formatiert oder unformatiert sein:

```
#-------------------------------------------------------------------
    --------
# getFrankingId
#
# Returns the Franking ID contained in the mentry widget win of type
```

```
# FrankingId, per default formatted as a string of the form
# "XX XXXX XXXX XX XXXX XXXX", where each X is an uppercase
    hexadecimal digit.
#-------------------------------------------------------------------------
    --------
proc getFrankingId {win {formatted 1}} {
    #
    # Check the widget
    #
    checkIfFrankingIdMentry $win

    #
    # Generate an error if any entry component is empty or incomplete.
    # "Incomplete" means in case of the last entry that it has less
    than
    # 3 characters, and in case of the other entries that it is not
    full.
    #
    for {set n 0} {$n < 6} {incr n} {
        if {[$win isempty $n]} {
            focus [$win entrypath $n]
            return -code error EMPTY
        }

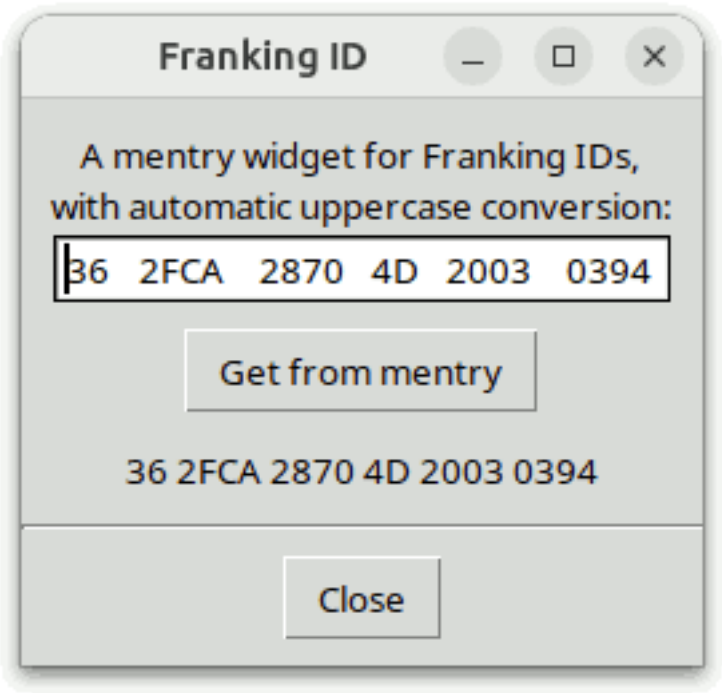
        if {$n == 5} {
            set w [$win entrypath $n]
            if {[string length [$w get]] < 3} {
                focus $w
                return -code error INCOMPL
            }
        } elseif {![$win isfull $n]} {
            focus [$win entrypath $n]
            return -code error INCOMPL
        }
    }

    #
    # Return the Franking ID built from the values contained in
    # the entry components and extended by the checksum if needed
    #
    set id [$win getstring]
    if {[string length $id] == 24} {  ;# i.e., last entry contains 3
    characters
        append id [crc4 $id]
    } elseif {[string index $id end] ne [crc4 [string range $id 0
    end-1]]} {
```

```
        focus $w
        return -code error BAD_CRC
    }
    return [expr {$formatted ? $id : [string map {" " ""} $id]}]
}
```

Wie im obigen Code zu sehen ist, berechnet die Prozedur die Prüfsumme und hängt sie an das Ergebnis an, wenn der letzte Eintrag nur 3 Zeichen enthält. Andernfalls prüft sie, ob das letzte Zeichen mit der aus den ersten 24 Zeichen (von denen nur 19 aussagekräftig sind) gebildeten Prüfsumme übereinstimmt, und löst einen Fehler aus, wenn dies nicht der Fall ist. (Beachten Sie, dass der Prüfsummenalgorithmus Leerzeichen ignoriert.)

Auch hier werden wir den restlichen Code nicht zeigen. Das Eingabe-Widget für Franking-IDs sieht wie im folgenden Screenshot aus:



Frankierung ID

Verwendung von Mentry-Widgets für Datum und Uhrzeit

Mit Widgets für mehrere Einträge lassen sich Datum und Uhrzeit in einer Vielzahl von Formaten anzeigen und bearbeiten. Das Mentry-Paket enthält hierfür gebrauchsfertige Befehle:

- Die Prozedur `mentry::dateMentry` erstellt ein neues Mentry-Widget zur Anzeige und Bearbeitung eines Datums gemäß dem Format, das dem Befehl als dreistelliger String übergeben wird, bestehend aus den Felddeskriptorzeichen `"d"`, `"m"` sowie `"y"` oder `"Y"`, die aus dem Befehl `clock format` bekannt sind. Ein weiteres Argument, das von der Prozedur erwartet wird, ist die Zeichenkette (in der Regel ein einzelnes Zeichen), die in den Trennzeichen zwischen den drei Komponenten des Datums angezeigt werden soll.
- Ebenso erstellt die Prozedur `mentry::timeMentry` ein neues Mentry-Widget zur Anzeige und Bearbeitung einer Uhrzeit gemäß dem Format, das dem Befehl als zwei- oder dreistelliger String übergeben wird, bestehend aus den folgenden Feldbeschreibungszeichen des Befehls `clock format`: `"H"` oder `"I"`, gefolgt von `"M"` und optional dem Buchstaben `"S"`. Ein

"H" als erstes Zeichen legt das Zeitformat "%H:%M" oder "%H:%M:%S" fest (d. h. mit einer Stunde zwischen 0 und 23), während der Buchstabe "I" für "%I:%M %p" oder "%I:%M:%S %p" steht (d. h. mit AM/PM-Angabe). Die Prozedur erwartet die Trennzeichenfolge (in der Regel das Zeichen ":") als weiteres Argument der Befehlszeile.

- Schließlich erstellt die Prozedur mentry::dateTimeentry ein neues Mentry-Widget zur Anzeige und Bearbeitung von Datum und Uhrzeit gemäß dem Format, das als 5- oder 6-stellige Zeichenkette an den Befehl übergeben wird, wobei die ersten drei Zeichen aus den Buchstaben "d" bestehen, "m", und "y" oder "Y" bestehen, gefolgt von zwei oder drei weiteren Feldbeschreibungszeichen des Befehls clock format, die wie folgt lauten müssen: "H" oder "I", dann "M" und optional der Buchstabe "S" lauten müssen. Die Prozedur erwartet zwei weitere Argumente, die die Trennzeichen festlegen, die jeweils im Datums- und im Zeitteil des "mentry"-Widgets verwendet werden sollen.

Bevor wir die weiteren Befehle des Mentry-Pakets beschreiben, die sich auf Datum und Uhrzeit beziehen, wollen wir uns zunächst ansehen, wie die ersten beiden der oben genannten Befehle in der Datei datetime1.tcl im Verzeichnis demos aufgerufen werden:

```
package require mentry

set title "Date & Time"
wm title . $title

#
# Add some entries to the Tk option database
#
source [file join [file dirname [info script]] option.tcl]

#
# Date and time formats supported by this demo
# script and the corresponding field separators
#
array set dateFmts {0 mdy  1 dmy  2 Ymd}
array set dateSeps {0 /    1 .    2 -  }
array set timeFmts {0 IMS  1 HMS}
array set timeSeps {0 :    1 :  }

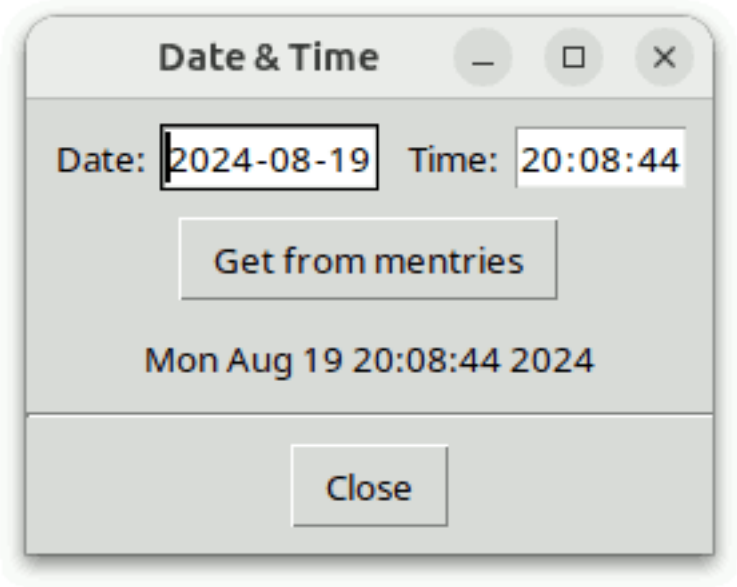
#
# Choose the date & time formats; don't use the %p field descriptor
# for displaying the AM/PM indicator, because it doesn't
# work on UNIX if Tcl/Tk is used in a non-default locale
#
wm withdraw .
set clockVal [clock seconds]
if {[clock format $clockVal -format "%H"] < 12} {
    set meridian AM
} else {
    set meridian PM
```

```
}
set dateIdx [tk_dialog .choice $title "Please choose a date format" {}
    -1 \
                        [clock format $clockVal -format "%m/%d/%y"] \
                        [clock format $clockVal -format "%d.%m.%y"] \
                        [clock format $clockVal -format "%Y-%m-%d"]]
set timeIdx [tk_dialog .choice $title "Please choose a time format" {}
    -1 \
                        [clock format $clockVal -format "%I:%M:%S
    $meridian"] \
                        [clock format $clockVal -format "%H:%M:%S"]]
wm deiconify .

#
# Frame .f with mentries displaying the date & time
#
frame .f
label .f.lDate -text "Date: "
mentry::dateMentry .f.date $dateFmts($dateIdx) $dateSeps($dateIdx) \
                   -justify center
frame .f.gap -width 10
label .f.lTime -text "Time: "
mentry::timeMentry .f.time $timeFmts($timeIdx) $timeSeps($timeIdx) \
                   -justify center
pack .f.lDate .f.date .f.gap .f.lTime .f.time -side left
```

Bevor das Hauptfenster angezeigt wird, lässt das Skript den Benutzer eines von drei Datums- und eines von zwei Zeitformaten auswählen. Die entsprechenden Befehlszeilenargumente, die an `mentry::dateMentry` und `mentry::timeMentry` übergeben werden, stammen aus den Arrays `dateFmts`, `dateSeps`, `timeFmts` und `timeSeps` entnommen.

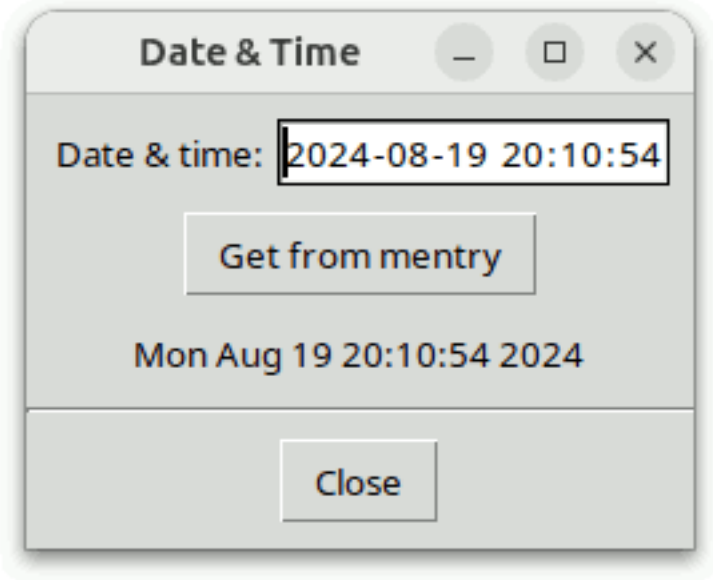
Die folgende Abbildung entspricht den Auswahlmöglichkeiten `dateIdx = 2` und `timeIdx = 1`:



Datum & Uhrzeit

Das Demo-Skript `datetime2.tcl` zeigt mithilfe des oben beschriebenen dritten Befehls sowohl das Datum als auch die Uhrzeit im selben Widget an:

```
#
# Frame .f with a mentry displaying the date & time
#
frame .f
label .f.l -text "Date & time: "
mentry::dateTimeMentry .f.me $dateFmts($dateIdx)$timeFmts($timeIdx) \
                       $dateSeps($dateIdx) $timeSeps($timeIdx) \
                       -justify center
pack .f.l .f.me
```



Datum & Uhrzeit

Das Mentry-Paket stellt zwei weitere Befehle für Datums-, Zeit- sowie Datums- und Zeit-Mentrys

bereit:

- Die Prozedur `mentry::putClockVal` gibt das Datum, die Uhrzeit oder Datum und Uhrzeit aus, die einem als erstes Argument festgelegten ganzzahligen Wert entsprechen, und übergibt diese Werte an ein Widget für Datum, Uhrzeit oder Datum und Uhrzeit, das als zweiter Parameter übergeben wird. Wie der Befehl `clock format` akzeptiert die Prozedur das optionale Argumentpaar `-gmt boolean`.
- Die Prozedur `mentry::getClockVal` gibt den Wert der Uhrzeit zurück, der dem Datum, der Uhrzeit oder dem Datum und der Uhrzeit entspricht, die in dem als erstes Argument angegebenen Datums-, Uhrzeit- oder Datums- und Uhrzeiteintrag enthalten sind. Wie der Befehl `clock scan` akzeptiert die Prozedur die optionalen Argumentpaare `-base clockValue` und `-gmt boolean`. Im Fehlerfall setzt die Prozedur den Fokus auf die erste fehlerhafte Eintragskomponente, löst einen Fehler aus und gibt einen der Werte zurück, die im folgenden Codefragment aus den Skripten `datetime1.tcl` und `datetime2.tcl` enthalten sind:

  Die Zeichenfolge `"EMPTY"` wird gegeben, wenn eine der Eintragskomponenten (mit Ausnahme derjenigen, die die Sekunden enthält) leer war. Der Wert `"BAD"` bedeutet, dass ein Tag-, Monats- oder Stundenwert von `0` vorliegt (die Stunde darf nicht `0` sein, wenn das AM/PM-Kennzeichen vorhanden ist). Die Zeichenkette `"BAD_DATE"` wird zurückgegeben, wenn das Tripel <JAHR, Monat, Tag> ungültig ist (beachten Sie, dass die Prozedur Schaltjahre berücksichtigt). Selbst wenn dieses Tripel gültig ist, kann die Konvertierung (die mit Hilfe des Befehls `clock scan` erfolgt) aufgrund eines nicht unterstützten Werts für das Jahr (z. B. zwischen `38` und `70`) fehlschlagen; in diesem Fall gibt die Prozedur die Zeichenkette `"BAD_YEAR"` zurück.

```
#
# Message strings corresponding to the values
# returned by mentry::getClockVal on failure
#
array set msgs {
    EMPTY       "Field value missing"
    BAD         "Invalid field value"
    BAD_DATE    "Invalid date"
    BAD_YEAR    "Unsupported year"
}
```

Das Demo-Skript `datetime1.tcl` führt die letzten beiden Befehle wie folgt aus:

```
#
# Button .get invoking the procedure mentry::getClockVal
#
button .get -text "Get from mentries" -command {
    if {[catch {
        set dateTime ""
        set base [mentry::getClockVal .f.date]
        set clockVal [mentry::getClockVal .f.time -base $base]
        set dateTime [clock format $clockVal -format "%c"]
    } result] != 0} {
```

```
        bell
        tk_messageBox -icon error -message $msgs($result) \
                      -title $title -type ok
    }
}

#
# Label .dateTime displaying the result of mentry::getClockVal
#
label .dateTime -textvariable dateTime

. . .

set clockVal [clock seconds]
mentry::putClockVal $clockVal .f.date
mentry::putClockVal $clockVal .f.time
focus [.f.date entrypath 0]
```

Um den Wert der Uhrzeit aus den Mentry-Widgets `.f.date` und `.f.time` zu erhalten, übergeben wir zunächst den Namen des Datums-Mentrys an den Befehl `mentry::getClockVal` und verwenden dann das Ergebnis als Wert der Option `-base`, wenn wir den Namen des Zeit-Mentrys an dieselbe Prozedur übergeben.

Das Demo-Skript `datetime2.tcl` ist einfacher:

```
#
# Button .get invoking the procedure mentry::getClockVal
#
button .get -text "Get from mentry" -command {
    if {[catch {
        set dateTime ""
        set clockVal [mentry::getClockVal .f.me]
        set dateTime [clock format $clockVal -format "%c"]
    } result] != 0} {
        bell
        tk_messageBox -icon error -message $msgs($result) \
                      -title $title -type ok
    }
}

#
# Label .dateTime displaying the result of mentry::getClockVal
#
label .dateTime -textvariable dateTime

. . .
```

```
set clockVal [clock seconds]
mentry::putClockVal $clockVal .f.me
focus [.f.me entrypath 0]
```

Auf Tile aufbauende Demo-Skripte

Die Mentry-Distribution enthält auch auf Tile aufbauende Entsprechungen der oben besprochenen Demo-Skripte. Wie im Abschnitt Mehr zu Mentry_tile dieses Tutorials beschrieben, ist es recht einfach, eine Anwendung, die das Mentry-Paket verwendet, auf eine auf Mentry_tile aufbauende Anwendung umzustellen. Sehen Sie sich zum Beispiel an, wie Sie das Demo-Skript `phonenumber.tcl` in ein auf Tile aufbauendes Skript namens `phonenumber_tile.tcl` umwandeln. Die Änderungen sind unten in der Farbe red dargestellt:

```
package require mentry_tile

set title "Phone Number"
wm title . $title

#
# Add some entries to the Tk option database
#
source [file join [file dirname [info script]] option_tile.tcl]

. . .

proc pastePhoneNumber w {

    . . .

    if {$res == 0} {
        set win [winfo parent [winfo parent $w]]
        . . .
    }
}

. . .

#
# Improve the window's appearance by using a tile
# frame as a container for the other widgets
#
ttk::frame .base

#
# Frame .base.f with a mentry displaying a phone number
#
ttk::frame .base.f
```

```
ttk::label .base.f.l -text "A mentry widget for phone numbers:"
phoneNumberMentry .base.f.me
pack .base.f.l .base.f.me

#
# Message strings corresponding to the values
# returned by getPhoneNumber on failure
#
array set msgs {
    EMPTY       "Field value missing"
    INCOMPL     "Incomplete field value"
}

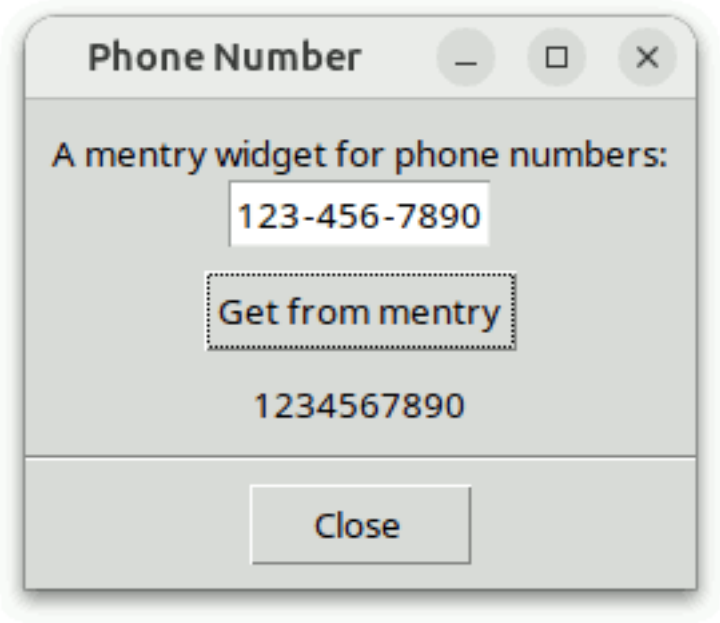
#
# Button .base.get invoking the procedure getPhoneNumber
#
ttk::button .base.get -text "Get from mentry" -command {
    if {[catch {
        set num ""
        set num [getPhoneNumber .base.f.me]
    } result] != 0} {
        bell
        tk_messageBox -icon error -message $msgs($result) \
                      -title $title -type ok
    }
}

#
# Label .base.num displaying the result of getPhoneNumber
#
ttk::label .base.num -textvariable num

. . .

putPhoneNumber 1234567890 .base.f.me
focus [.base.f.me entrypath 0]
```

Das war's schon! Das daraufhin angezeigte Fenster hat ein ansprechendes, themenspezifisches Erscheinungsbild:



Telefonnummer

Die einzigen für Mentry spezifischen Änderungen im obigen Code bestehen in der Verwendung des Suffixes `_tile` in `mentry_tile` und einem zweiten Aufruf von `winfo parent` in der Prozedur `pastePhoneNumber` (aus technischen Gründen sind die Komponenten eines auf Tile aufbauenden Mentry-Widgets *Enkel* des Mentry).

mentryDateTime

Stichwörter: mentryDateTime

Mehrfeld-Eingabe-Widgets für Datum und Uhrzeit

Für Mentry Version 4.5

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Der Befehl mentry::dateMentry
- Der Befehl mentry::timeMentry
- Der Befehl mentry::dateTimeMentry
- Der Befehl mentry::putClockVal
- Der Befehl mentry::getClockVal

Der Befehl `mentry::dateMentry`

NAME

`mentry::dateMentry` – Mentry-Widgets für Datumsangaben erstellen und bearbeiten

ÜBERSICHT

```
mentry::dateMentry pathName format separator ?options?
```

BESCHREIBUNG

Dieser Befehl erstellt ein neues Mentry-Widget `pathName` zur Anzeige und Bearbeitung eines Datums gemäß dem Argument `format`, bei dem es sich um eine Zeichenkette der Länge 3 handeln muss, bestehend aus den Buchstaben `"d"` für den Tag (`01` – `31`), `"m"` für den Monat (`01` – `12`) sowie `"y"` oder `"Y"` für das Jahr ohne Jahrhundertangabe (`00` – `99`) oder mit Jahrhundertangabe (z. B. `2008`) in beliebiger Reihenfolge. Diese Feldbeschreibungszeichen haben dieselbe Bedeutung wie im Befehl `clock format`. Das Argument `separator` legt den Text fest, der in den Beschriftungen angezeigt werden soll, die die drei Eingabekomponenten des mentry-Widgets voneinander trennen (die gängigsten Werte sind die Zeichen `"/"`, `"-"` und `"."`). Die unterstützten `options` entsprechen denen des Befehls `mentry::mentry`.

Der Befehl setzt das Attribut `type` des Widgets auf den Wert `"Date"`, speichert den Wert von `format` in dessen Attribut `format` und gibt den Namen des neu erstellten Widgets zurück.

VORGABEBINDUNGEN

Der Befehl `mentry::dateMentry` definiert vier neue Bindungen für die Eingabekomponenten des von ihm erstellten "mentry"-Widgets: Die Taste `Up` erhöht den Wert der Eingabe um 1, wenn dieser unter dem für diese Komponente zulässigen Höchstwert liegt. Ebenso verringert die Taste `Down` den Wert des Eingabefelds um 1, wenn dieser größer ist als der zulässige Mindestwert für diese Komponente. Die Taste `Prior` erhöht den Wert des Eintrags um höchstens 10, wenn dieser kleiner als das für diese Komponente zulässige Maximum ist. Ebenso verringert die Taste `Next` den Wert des Eintrags um höchstens 10, wenn dieser größer ist als der zulässige Mindestwert für diese Komponente. Ist der Eintrag leer, fügen alle diese Tasten den komponenten-spezifischen Mindestwert in den Eintrag ein.

Die von den Tasten `Up` und `Down` ausgeführten Aktionen können auch mithilfe des Mausrads oder einer Zwei-Finger-Geste auf dem Touchpad ausgelöst werden. Darüber hinaus werden unter Mac OS X/11+ Aqua die von den Tasten `Prior` und `Next` ausgeführten Aktionen auch durch das Ereignis `<Option-MouseWheel>` ausgelöst. Das Gleiche gilt für alle Fenstersysteme, wenn die Tk-Version 8.7a4 oder höher ist, wobei der Modifikator `Option` unter Mac OS X/11+ der Taste `Option` und unter Windows sowie X11 der Taste `Alt` zugeordnet ist.

SCHLÜSSELWÖRTER

mentry, widget, date

Der Befehl `mentry::timeMentry`

NAME

`mentry::timeMentry` – Mentry-Widgets für die Zeit erstellen und bearbeiten

ÜBERSICHT

```
mentry::timeMentry pathName format separator ?options?
```

BESCHREIBUNG

Dieser Befehl erstellt ein neues Mentry-Widget `pathName` zur Anzeige und Bearbeitung einer Uhrzeit gemäß dem Argument `format`, bei dem es sich um eine Zeichenkette der Länge 2 oder 3 handeln muss, bestehend aus den folgenden Feldbeschreibungszeichen des Befehls `clock format`: `"H"` oder `"I"`, gefolgt von `"M"` und optional dem Buchstaben `"S"`. Ein `"H"` als erstes Zeichen legt das Zeitformat `"%H:%M"` oder `"%H:%M:%S"` fest (d. h. mit der Stunde zwischen `0` und `23`), während der Buchstabe `"I"` für `"%I:%M %p"` oder `"%I:%M:%S %p"` steht (d. h. mit AM/PM-Angabe). Das Argument `separator` legt den Text fest, der in den Beschriftungen angezeigt werden soll, die die Eingabekomponenten des mentry-Widgets voneinander trennen (dies ist in der Regel das Zeichen `":"`). Die `options` entsprechen denen des Befehls `mentry::mentry`.

Der Befehl setzt das Attribut `type` des Widgets auf den Wert `"Time"`, speichert den Wert von `format` in dessen Attribut `format` und gibt den Namen des neu erstellten Widgets zurück.

VORGABEBINDUNGEN

Der Befehl `mentry::timeMentry` definiert vier neue Tastaturbindungen für die Eingabekomponenten des von ihm erstellten "mentry"-Widgets: Die Taste `Up` erhöht den Wert der Eingabe um 1, wenn dieser unter dem für diese Komponente zulässigen Höchstwert liegt. Ebenso verringert die Taste `Down` den Wert des Eingabefelds um 1, wenn dieser größer ist als der zulässige Mindestwert für diese Komponente. Die Taste `Prior` erhöht den Wert des Eintrags um höchstens 10, wenn dieser kleiner als der zulässige Höchstwert für diese Komponente ist. Ebenso verringert die Taste `Next` den Wert des Eintrags um höchstens 10, wenn dieser größer ist als der zulässige Mindestwert für diese Komponente. Ist der Eintrag leer, fügen alle diese Tasten den komponentenbezogenen Mindestwert in den Eintrag ein.

Die von den Tasten `Up` und `Down` ausgeführten Aktionen können auch mithilfe des Mausrads oder einer Zwei-Finger-Geste auf dem Touchpad ausgelöst werden. Darüber hinaus werden unter Mac OS X/11+ Aqua die von den Tasten `Prior` und `Next` ausgeführten Aktionen auch durch das Ereignis `<Option-MouseWheel>` ausgelöst. Das Gleiche gilt für alle Fenstersysteme, wenn die Tk-Version 8.7a4 oder höher ist, wobei der Modifikator `Option` unter Mac OS X/11+ der Taste `Option` und unter Windows sowie X11 der Taste `Alt` zugeordnet ist.

SCHLÜSSELWÖRTER

mentry, widget, time

Der Befehl `mentry::dateTimeMentry`

NAME

`mentry::dateTimeMentry` – Mentry-Widgets für Datum und Uhrzeit erstellen und bearbeiten

ÜBERSICHT

```
mentry::dateTimeMentry pathName format dateSeparator timeSeparator
    ?options?
```

BESCHREIBUNG

Dieser Befehl erstellt ein neues Mentry-Widget `pathName` zur Anzeige und Bearbeitung von

Datum und Uhrzeit gemäß dem Argument `format`, das eine Zeichenkette mit einer Länge von 5 oder 6 Zeichen sein muss, wobei die ersten 3 Zeichen aus den Buchstaben `"d"` für den Tag (`01` – `31`), `"m"` für den Monat (`01` – `12`) sowie `"y"` oder `"Y"` für das Jahr ohne Jahrhundertangabe (`00` – `99`) oder mit Jahrhundertangabe (z. B. `2008`), in beliebiger Reihenfolge, gefolgt von 2 oder 3 weiteren Feldbeschreibungszeichen des Befehls `clock format`, die wie folgt lauten müssen: `"H"` oder `"I"`, dann `"M"` und optional der Buchstabe `"S"`. Ein `"H"` legt das Zeitformat `"%H:%M"` oder `"%H:%M:%S"` fest (d. h. mit der Stunde zwischen `0` und `23`), während der Buchstabe `"I"` für `"%I:%M %p"` oder `"%I:%M:%S %p"` steht (d. h. mit AM/PM-Angabe). Das Argument `dateSeparator` legt den Text fest, der in den Beschriftungen angezeigt werden soll, die die Eingabekomponenten im Datumsbereich des Eingabe-Widgets voneinander trennen (die gängigsten Werte sind die Zeichen `"/"`, `"-"` und `"."`). Ebenso steht das Argument `timeSeparator` für den Text, der in den Beschriftungen angezeigt werden soll, die die Eingabekomponenten im Zeitteil des mentry-Widgets trennen (dies ist in der Regel das Zeichen `":"`). Der Datums- und der Zeitteil sind wiederum durch ein Leerzeichen voneinander getrennt. Die unterstützten `options` entsprechen denen des Befehls `mentry::mentry`.

Der Befehl setzt das Attribut `type` des Widgets auf den Wert `"DateTime"`, speichert den Wert von `format` in dessen Attribut `format` und gibt den Namen des neu erstellten Widgets zurück.

VORGABEBINDUNGEN

Der Befehl `mentry::dateTimeMentry` definiert vier neue Bindungen für die Eingabekomponenten des von ihm erstellten "mentry"-Widgets: Die Taste `Up` erhöht den Wert der Eingabe um 1, wenn dieser unter dem für diese Komponente zulässigen Höchstwert liegt. Ebenso verringert die Taste `Down` den Wert des Eingabefelds um 1, wenn dieser größer ist als der zulässige Mindestwert für diese Komponente. Die Taste `Prior` erhöht den Wert des Eintrags um höchstens 10, wenn dieser kleiner als der zulässige Höchstwert für diese Komponente ist. Ebenso verringert die Taste `Next` den Wert des Eintrags um höchstens 10, wenn dieser größer als das zulässige Minimum für diese Komponente ist. Wenn der Eintrag leer ist, fügen alle diese Tasten den komponentenbezogenen Mindestwert in den Eintrag ein.

Die von den Tasten `Up` und `Down` ausgeführten Aktionen können auch mithilfe des Mausrads oder einer Zwei-Finger-Geste auf dem Touchpad ausgelöst werden. Darüber hinaus werden unter Mac OS X/11+ Aqua die von den Tasten `Prior` und `Next` ausgeführten Aktionen auch durch das Ereignis `<Option-MouseWheel>` ausgelöst. Das Gleiche gilt für alle Fenstersysteme, wenn die Tk-Version 8.7a4 oder höher ist, wobei der Modifikator `Option` unter Mac OS X/11+ der Taste `Option` und unter Windows sowie X11 der Taste `Alt` zugeordnet ist.

SCHLÜSSELWÖRTER

mentry, widget, date, time

Der Befehl `mentry::putClockVal`

NAME

`mentry::putClockVal` – Ausgabe eines Werts für Datum, Uhrzeit oder Datum und Uhrzeit

ÜBERSICHT

```
mentry::putClockVal clockValue pathName ?-gmt boolean?
```

BESCHREIBUNG

Dieser Befehl gibt das Datum, die Uhrzeit oder Datum und Uhrzeit aus, die der Ganzzahl `clockValue` entsprechen, und überträgt sie an das Mentry-Widget `pathName`, das zuvor mit dem Befehl `mentry::dateMentry`, `mentry::timeMentry` oder `mentry::dateTimeMentry` erstellt worden sein muss (dies wird durch Überprüfung des Attributs `type` des Widgets überprüft, das den Wert `"Date"`, `"Time"` oder `"DateTime"` haben muss).

Wie beim Befehl `clock format` legt das optionale Argumentpaar `-gmt boolean` fest, ob der Wert der Zeit als Greenwich Mean Time oder gemäß der von der Betriebsumgebung definierten lokalen Zeitzone formatiert werden soll.

SCHLÜSSELWÖRTER

mentry, widget, date, time, clock

Der Befehl `mentry::getClockVal`

NAME

`mentry::getClockVal` – Den Wert der Uhrzeit aus einem Datums-, Zeit- oder Datums- und Zeit-Eintrag abrufen

ÜBERSICHT

```
mentry::getClockVal pathName ?-base clockValue? ?-gmt boolean?
```

BESCHREIBUNG

Dieser Befehl gibt den Uhrzeitwert zurück, der dem Datum, der Uhrzeit oder dem Datum und der Uhrzeit entspricht, die im Mentry-Widget `pathName` enthalten sind, das mit dem Befehl `mentry::dateMentry`, `mentry::timeMentry` oder `mentry::dateTimeMentry` erstellt worden sein muss (dies wird durch Überprüfung des Attributs `type` des Widgets überprüft, das den Wert `"Date"`, `"Time"` oder `"DateTime"` haben muss).

Wie beim Befehl `clock scan` gilt: legt das optionale Argumentpaar `-base clockValue` fest, dass das in `clockValue` enthaltene Datum verwendet werden soll, wenn der Uhrzeitwert aus dem angegebenen Mentry-Widget abgerufen wird, und das optionale Argumentpaar `-gmt boolean` legt fest, ob der Uhrzeitwert relativ zur Greenwich Mean Time oder gemäß der von der Betriebsumgebung definierten lokalen Zeitzone berechnet werden soll.

Tritt während der Konvertierung ein Fehler auf, setzt der Befehl den Fokus auf die erste fehlerhafte Eingabekomponente, löst einen Fehler aus und gibt einen der in der folgenden Tabelle aufgeführten Werte zurück:

Rückgabewert	Bedeutung
"EMPTY"	Jede Eingabekomponente (mit Ausnahme derjenigen, die die Sekunden enthält) ist leer.

Rückgabewert	Bedeutung
"BAD"	Der Wert für den Tag, den Monat oder die Stunde (letzteres nur, wenn die AM/PM-Angabe vorhanden ist) lautet 0.
"BAD_DATE"	Das Tripel <Jahr, Monat, Tag> ist ungültig (beachten Sie, dass der Befehl Schaltjahre berücksichtigt).
"BAD_YEAR"	Das obige Tripel ist gültig, aber die Konvertierung (die mit Hilfe des Befehls clock scan durchgeführt wurde) ist aufgrund eines nicht unterstützten Werts für das Jahr fehlgeschlagen (z. B. zwischen 38 und 70).

SCHLÜSSELWÖRTER

mentry, widget, date, time, clock

mentryFixedPoint

Stichwörter: mentryFixedPoint

Mehrfeld-Eingabe-Widgets für reelle Zahlen

im Festkommaformat

Für Mentry Version 4.5

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Der Befehl mentry::fixedPointMentry
- Der Befehl mentry::putReal
- Der Befehl mentry::getReal

Der Befehl `mentry::fixedPointMentry`

NAME

`mentry::fixedPointMentry` – Erstellen und Bearbeiten von mentry-Widgets für reelle Zahlen im Festkommaformat

ÜBERSICHT

```
mentry::fixedPointMentry pathName count1 count2 ?-comma? ?options?
```

BESCHREIBUNG

Dieser Befehl erstellt ein neues mentry-Widget `pathName` zur Anzeige und Bearbeitung von reellen Zahlen im Festkommaformat, mit `count1` Zeichen vor und `count2` Ziffern nach dem Dezimalpunkt. Sowohl `count1` als auch `count2` müssen positive ganze Zahlen sein. Das Mentry verfügt über zwei Eingabekomponenten, die standardmäßig durch eine Beschriftung

mit einem `"."`-Zeichen getrennt sind; der optionale Schalter `-comma` ändert den Beschriftungstext in `","`. Die unterstützten `options` entsprechen denen des Befehls `mentry::mentry`.

Der Befehl setzt das Attribut `type` des Widgets auf den Wert `"FixedPoint"` und gibt den Namen des neu erstellten Widgets zurück.

SCHLÜSSELWÖRTER

mentry, widget, real number

Der Befehl `mentry::putReal`

NAME

`mentry::putReal` – Eine reelle Zahl in ein Mentry vom Typ `"FixedPoint"` ausgeben

ÜBERSICHT

```
mentry::putReal number pathName
```

BESCHREIBUNG

Dieser Befehl gibt die reelle Zahl `number` an das mentry-Widget `pathName` aus, das zuvor mit dem Befehl `mentry::fixedPointMentry` erstellt worden sein muss (dies wird durch Überprüfung des Attributs `type` des Widgets überprüft, das den Wert `"FixedPoint"` haben muss).

Der Befehl löst einen Fehler aus, wenn die Zahl nicht in das Widget passt, weil sie zu viele Zeichen vor dem Dezimalpunkt enthält.

SCHLÜSSELWÖRTER

mentry, widget, real number

Der Befehl `mentry::getReal`

NAME

`mentry::getReal` – Eine reelle Zahl aus einem Mentry des Typs `"FixedPoint"` abrufen

ÜBERSICHT

```
mentry::getReal pathName
```

BESCHREIBUNG

Dieser Befehl gibt die Zahl zurück, die im Mentry-Widget `pathName` enthalten ist, das mit dem Befehl `mentry::fixedPointMentry` erstellt worden sein muss (dies wird durch Überprüfung des Attributs `type` des Widgets überprüft, das den Wert `"FixedPoint"` haben muss).

Wenn beide Eingabekomponenten des Widgets leer sind, setzt der Befehl den Fokus auf die erste Eingabekomponente, löst einen Fehler aus und gibt die Zeichenfolge `"EMPTY"` zurück.

SCHLÜSSELWÖRTER

mentry, widget, real number

mentryIPAddr

Stichwörter: mentryIPAddr

Mehrfeld-Eingabe-Widgets für IP-Adressen

Für Mentry Version 4.5

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Der Befehl mentry::ipAddrMentry
- Der Befehl mentry::putIPAddr
- Der Befehl mentry::getIPAddr

Der Befehl `mentry::ipAddrMentry`

NAME

`mentry::ipAddrMentry` – Mentry-Widgets für IP-Adressen erstellen und bearbeiten

ÜBERSICHT

```
mentry::ipAddrMentry pathName ?options?
```

BESCHREIBUNG

Dieser Befehl erstellt ein neues Mentry-Widget `pathName` zur Anzeige und Bearbeitung von IP-Adressen in der standardmäßigen Dezimalschreibweise mit Punkttrennung. Die unterstützten `options` entsprechen denen des Befehls `mentry::mentry`.

Der Befehl setzt das Attribut `type` des Widgets auf den Wert `"IPAddr"` und gibt den Namen des neu erstellten Widgets zurück.

VORGABEBINDUNGEN

Der Befehl `mentry::ipAddrMentry` definiert fünf neue Tastaturbindungen für die Eingabekomponenten des von ihm erstellten "mentry"-Widgets: Die Taste `Up` erhöht den Wert des Eingabefelds um 1, wenn dieser kleiner als 255 ist. Ebenso verringert die Taste `Down` den Wert des Eingabefelds um 1, wenn dieser größer als 0 ist. Die Taste `Prior` erhöht den Wert des Eintrags um höchstens 10, wenn dieser kleiner als 255 ist. Ebenso verringert die Taste `Next` den Wert des Eintrags um höchstens 10, wenn dieser größer als 0 ist. Ist der Eintrag leer, fügen alle diese Tasten den Wert `0` in den Eintrag ein. Schließlich fügt `Control-v` (`Command-v` auf dem Mac) den aktuellen Inhalt der Zwischenablage in den Mentry ein, sofern es sich um eine gültige IP-Adresse handelt.

Die von den Tasten `Up` und `Down` ausgeführten Aktionen können auch mithilfe des Mausrads oder einer Zwei-Finger-Geste auf dem Touchpad ausgelöst werden. Darüber hinaus werden unter Mac OS X/11+ Aqua die von den Tasten `Prior` und `Next` ausgeführten Aktionen auch

durch das Ereignis `<Option-MouseWheel>` ausgelöst. Das Gleiche gilt für alle Fenstersysteme, wenn die Tk-Version 8.7a4 oder höher ist, wobei der Modifikator `Option` unter Mac OS X/11+ der Taste `Option` und unter Windows sowie X11 der Taste `Alt` zugeordnet ist.

SCHLÜSSELWÖRTER

mentry, widget, IP address

Der Befehl `mentry::putIPAddr`

NAME

`mentry::putIPAddr` – Eine IP-Adresse in eine Mentry vom Typ `"IPAddr"` ausgeben

ÜBERSICHT

```
mentry::putIPAddr address pathName
```

BESCHREIBUNG

Dieser Befehl gibt die IP-Adresse `address` an das Mentry-Widget `pathName` aus, das zuvor mit dem Befehl `mentry::ipAddrMentry` erstellt worden sein muss (dies wird durch Überprüfung des Attributs `type` des Widgets überprüft, das den Wert `"IPAddr"` haben muss).

Der Befehl löst einen Fehler aus, wenn die Adresse ungültig ist.

SCHLÜSSELWÖRTER

mentry, widget, IP address

Der Befehl `mentry::getIPAddr`

NAME

`mentry::getIPAddr` – Eine IP-Adresse aus einer Mentry vom Typ `"IPAddr"` abrufen

ÜBERSICHT

```
mentry::getIPAddr pathName
```

BESCHREIBUNG

Dieser Befehl gibt die IP-Adresse zurück, die im Mentry-Widget `pathName` enthalten ist, das mit dem Befehl `mentry::ipAddrMentry` erstellt worden sein muss (dies wird durch Überprüfung des Attributs `type` des Widgets überprüft, das den Wert `"IPAddr"` haben muss).

Ist eine der Eingabekomponenten des Widgets leer, setzt der Befehl den Fokus auf die erste dieser Eingaben, löst einen Fehler aus und gibt die Zeichenfolge `"EMPTY"` zurück.

SCHLÜSSELWÖRTER

mentry, widget, IP address

mentryIPv6Addr

Stichwörter: mentryIPv6Addr

Mehrfeld-Eingabe-Widgets für IPv6-Adressen

Für Mentry Version 4.5

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Der Befehl mentry::ipv6AddrMentry
- Der Befehl mentry::putIPv6Addr
- Der Befehl mentry::getIPv6Addr

Der Befehl `mentry::ipv6AddrMentry`

NAME

`mentry::ipv6AddrMentry` – Erstellen und Bearbeiten von Mentry-Widgets für IPv6-Adressen

ÜBERSICHT

```
mentry::ipv6AddrMentry pathName ?options?
```

BESCHREIBUNG

Dieser Befehl erstellt ein neues Mentry-Widget `pathName` zur Anzeige und Bearbeitung von IPv6-Adressen in der standardmäßigen hexadezimalen Notation (wobei die acht Gruppen aus jeweils einer bis vier hexadezimalen Ziffern durch jeweils einen Doppelpunkt voneinander getrennt sind). Die unterstützten `options` entsprechen denen des Befehls `mentry::mentry`.

Der Befehl setzt das Attribut `type` des Widgets auf den Wert `"IPv6Addr"` und gibt den Namen des neu erstellten Widgets zurück.

VORGABEBINDUNGEN

Der Befehl `mentry::ipv6AddrMentry` definiert fünf neue Bindungen für die Eingabekomponenten des von ihm erstellten "mentry"-Widgets: Die Taste `Up` erhöht den Wert des Eingabefelds um 1, wenn dieser kleiner als 65535 ist. Ebenso verringert die Taste `Down` den Wert des Eintrags um 1, wenn dieser größer als 0 ist. Die Taste `Prior` erhöht den Wert des Eintrags um höchstens 10, wenn dieser kleiner als 65535 ist. Ebenso verringert die Taste `Next` den Wert des Eintrags um höchstens 10, wenn dieser größer als 0 ist. Ist der Eintrag leer, fügen alle diese Tasten den Wert `0` in den Eintrag ein. Schließlich fügt `Control-v` (`Command-v` auf dem Mac) den aktuellen Inhalt der Zwischenablage in den Mentry ein, sofern es sich um eine gültige IPv6-Adresse handelt.

Die von den Tasten `Up` und `Down` ausgeführten Aktionen können auch mithilfe des Mausrads oder einer Zwei-Finger-Geste auf dem Touchpad ausgelöst werden. Darüber hinaus werden unter Mac OS X/11+ Aqua die von den Tasten `Prior` und `Next` ausgeführten Aktionen auch durch das Ereignis `<Option-MouseWheel>` ausgelöst. Das Gleiche gilt für alle

Fenstersysteme, wenn die Tk-Version 8.7a4 oder höher ist, wobei der Modifikator `Option` unter Mac OS X/11+ der Taste `Option` und unter Windows sowie X11 der Taste `Alt` zugeordnet ist.

SCHLÜSSELWÖRTER

mentry, widget, IPv6 address

Der Befehl `mentry::putIPv6Addr`

NAME

`mentry::putIPv6Addr` – Eine IPv6-Adresse in ein Mentry des Typs `"IPv6Addr"` ausgeben

ÜBERSICHT

```
mentry::putIPv6Addr address pathName
```

BESCHREIBUNG

Dieser Befehl gibt die IPv6-Adresse `address` an das Mentry-Widget `pathName` aus, das zuvor mit dem Befehl `mentry::ipv6AddrMentry` erstellt worden sein muss (dies wird durch Überprüfung des Attributs `type` des Widgets überprüft, das den Wert `"IPv6Addr"` haben muss).

Die Adresse muss aus bis zu acht durch Doppelpunkte getrennten Gruppen bestehen, jede aus einer bis vier Hexadezimalziffern. Bei weniger als acht Gruppen muss die Adresse genau einmal `::` enthalten (das steht für aufeinanderfolgende Gruppen von Nullen). Der Befehl löst einen Fehler aus, wenn die Adresse ungültig ist.

SCHLÜSSELWÖRTER

mentry, widget, IPv6 address

Der Befehl `mentry::getIPv6Addr`

NAME

`mentry::getIPv6Addr` – Eine IPv6-Adresse aus einer Mentry vom Typ `"IPv6Addr"` abrufen

ÜBERSICHT

```
mentry::getIPv6Addr pathName
```

BESCHREIBUNG

Dieser Befehl gibt die IPv6-Adresse zurück, die im Mentry-Widget `pathName` enthalten ist, das mit dem Befehl `mentry::ipv6AddrMentry` erstellt worden sein muss (dies wird durch Überprüfung des Attributs `type` des Widgets überprüft, das den Wert `"IPv6Addr"` haben muss).

Ist eine der Eingabekomponenten des Widgets leer, setzt der Befehl den Fokus auf die erste dieser Eingaben, löst einen Fehler aus und gibt die Zeichenfolge `"EMPTY"` zurück.

SCHLÜSSELWÖRTER

mentry, widget, IPv6 address

mentryThemes

Stichwörter: mentryThemes

Befehle zu Tile-Themen

Für Mentry Version 4.5

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Übersicht
- Der Befehl mentry::setTheme
- Der Befehl mentry::getCurrentTheme
- Der Befehl mentry::getThemes
- Der Befehl mentry::setThemeDefaults

Überblick

Die auf dieser Referenzseite beschriebenen Befehle sollten nur bei Verwendung des Pakets "Mentry_tile" aufgerufen werden. Mit ihnen können Sie das aktuelle Thema festlegen und abfragen, eine Liste der verfügbaren Themen abrufen und sicherstellen, dass Ihre Widgets ein themenspezifisches Erscheinungsbild erhalten.

Der Befehl `mentry::setTheme`

NAME

`mentry::setTheme` – Aktuelles Thema festlegen

ÜBERSICHT

```
mentry::setTheme theme
```

BESCHREIBUNG

Dieser Befehl legt das aktuelle Thema auf `theme` fest und lädt es gegebenenfalls. Es handelt sich dabei lediglich um einen Alias für `ttk::setTheme` oder `tile::setTheme`, je nachdem, welche Tile-Version in den Interpreter geladen wurde. (Der Befehl `tile::setTheme` wurde in der Tile-Version 0.8 in `ttk::setTheme` umbenannt.)

Da es sich lediglich um einen Alias für eine Prozedur der Tile-Bibliothek handelt, führt der Befehl `mentry::setTheme` genau dasselbe aus wie der ursprüngliche Befehl: Er lädt bei Bedarf das Paket, das das angegebene Thema implementiert, legt das angegebene Thema fest und speichert dieses je nach aktueller Tile-Version in der Variable `ttk::currentTheme` oder `tile::currentTheme`.

SCHLÜSSELWÖRTER

mentry, theme, tile

Der Befehl `mentry::getCurrentTheme`

NAME

`mentry::getCurrentTheme` – Aktuelles Thema abrufen

ÜBERSICHT

```
mentry::getCurrentTheme
```

BESCHREIBUNG

Dieser Befehl gibt den Namen des aktuellen Themas zurück, indem er den Befehl `ttk::style theme use` aufruft. Wird eine ältere Tile-Version verwendet, die diese Methode noch nicht unterstützt, gibt der Befehl je nach Tile-Version den Wert der Variablen `ttk::currentTheme` oder `tile::currentTheme` zurück. (Der Namespace, der die Variable `currentTheme` enthält, wurde in der Tile-Version 0.8 von `tile` in `ttk` geändert.)

SCHLÜSSELWÖRTER

mentry, theme, tile

Der Befehl `mentry::getThemes`

NAME

`mentry::getThemes` – Die in der Paketdatenbank registrierten Themen abrufen

ÜBERSICHT

```
mentry::getThemes
```

BESCHREIBUNG

Dieser Befehl gibt eine Liste der in der Paketdatenbank registrierten Themen zurück. Es handelt sich dabei lediglich um einen Alias für `ttk::themes` oder `tile::availableThemes`, je nachdem, welche Tile-Version in den Interpreter geladen wurde. (Der Befehl `tile::availableThemes` wurde in der Tile-Version 0.8 in `ttk::themes` umbenannt.)

SCHLÜSSELWÖRTER

mentry, theme, tile

Der Befehl `mentry::setThemeDefaults`

NAME

`mentry::setThemeDefaults` – Festlegen themenspezifischer Standardwerte für bestimmte Mentry-Konfigurationsoptionen

ÜBERSICHT

```
mentry::setThemeDefaults
```

BESCHREIBUNG

Dieser Befehl füllt das Array `mentry::themeDefaults` mit themenspezifischen Standardwerten für eine Reihe von Mentry-Konfigurationsoptionen. Die Array-Namen entsprechen den Befehlszeilennamen der Optionen, und die zugehörigen Array-Werte sind die

Standardwerte dieser Konfigurationsoptionen für das aktuell festgelegte Tile-Thema.

Die Optionen, deren Namen und Werte in das Array `mentry::themeDefaults` geschrieben werden, sind: `-background`, `-foreground` und `-font`. Darüber hinaus setzt der Befehl einige weitere Array-Elemente auf themenspezifische Standardwerte, die für interne Zwecke benötigt werden, wie beispielsweise die Aktualisierung der Hintergrund- und Vordergrundfarben der Beschriftungskomponenten eines Mentry-Widgets im Zustand `disabled` oder `readonly`; die entsprechenden Array-Namen lauten: `-disabledbackground`, `-disabledforeground` und `-readonlybackground`. Schließlich werden die Array-Werte, die den Namen `-selectbackground`, `-selectforeground` und `-selectborderwidth` zugeordnet sind, von Mentry_tile nicht verwendet, könnten sich jedoch, wie im Folgenden beschrieben, für andere Zwecke als nützlich erweisen.

Der Befehl `mentry::setThemeDefaults` wird von Mentry_tile automatisch beim Start der Anwendung sowie dann aufgerufen, wenn das virtuelle Ereignis `<<ThemeChanged>>` an das Hauptfenster gesendet wird. Er wird außerdem unter macOS 10.14 (Mojave) und höher immer dann aufgerufen, wenn das Hauptfenster ein virtuelles Ereignis vom Typ `<<LightAppearance>>` (früher: `<<LightAqua>>`) oder `<<DarkAppearance>>` (früher: `<<DarkAqua>>`) erhält, wenn das aktuelle Thema `aqua` ist. In diesen Fällen sendet Mentry_tile nach dem Aufruf des Befehls `mentry::setThemeDefaults` das virtuelle Ereignis `<<MentryThemeDefaultsChanged>>` an das Hauptfenster und konfiguriert alle Mentry-Widgets der Anwendung neu, wobei die neuen Standardwerte jener Optionen verwendet werden, die nicht explizit auf andere Werte als die entsprechenden Standardwerte gesetzt wurden.

Unter macOS 10.14 (Mojave) und höher werden die virtuellen Ereignisse `<<LightAppearance>>` und `<<DarkAppearance>>` (bzw. `<<LightAqua>>` und `<<DarkAqua>>`) von Tk 8.6.10 und höher an alle Toplevel-Fenster gesendet, sobald das Systemdesign zwischen hellem und dunklem Modus wechselt oder sich die Akzentfarbe in den allgemeinen Einstellungen ändert. Tk-Versionen ab 8.6.11 senden diese Ereignisse unter macOS 10.15 (Catalina) und höher auch wenn die Hervorhebungsfarbe in den allgemeinen Einstellungen geändert wird.

Dieser Befehl wird nicht nur von Mentry_tile verwendet, sondern kann auch im Anwendungscode aufgerufen werden, obwohl es bei den aktuellen Mentry_tile-Versionen nicht notwendig ist, ihn explizit aufzurufen.

Durch die Verwendung der im Array `mentry::themeDefaults` enthaltenen Standardwerte können Sie sicherstellen, dass klassische Tk-Widgets, zum Beispiel text, genau wie die Tile-Widgets ein themenspezifisches Erscheinungsbild erhalten. So können Sie beispielsweise einige gängige Konfigurationsoptionen wie folgt zur Optionsdatenbank hinzufügen:

```
if {[tk windowingsystem] eq "x11"} {
    option add *Text.background   $mentry::themeDefaults(-background)
    option add *Text.foreground   $mentry::themeDefaults(-foreground)
    option add *selectBackground
    $mentry::themeDefaults(-selectbackground)
    option add *selectForeground
    $mentry::themeDefaults(-selectforeground)
}
option add *selectBorderWidth
    $mentry::themeDefaults(-selectborderwidth)
option add *Font                  $mentry::themeDefaults(-font)
```

Sie können außerdem das virtuelle Ereignis `<<MentryThemeDefaultsChanged>>` nutzen, um das Erscheinungsbild Ihrer Tk-Widgets entsprechend den neuen Werten im Array `mentry::themeDefaults` anzupassen, indem Sie eine Bindung an das dafür vorgesehene Bindungs-Tag `MentryMain` herstellen, das dem Hauptfenster zugeordnet ist:

```
bind MentryMain <<MentryThemeDefaultsChanged>> ...
```

Das Bindungs-Tag `MentryMain` wird vom Mentry_tile dem Hauptfenster zugewiesen und intern zur Verarbeitung der virtuellen Ereignisse `<<ThemeChanged>>`, `<<LightAppearance>>` (oder `<<LightAqua>>`) sowie `<<DarkAppearance>>` (oder `<<DarkAqua>>`). Wie oben gezeigt, können Anwendungen davon Gebrauch machen, z. B. zur Verarbeitung des virtuellen Ereignisses `<<MentryThemeDefaultsChanged>>`.

SCHLÜSSELWÖRTER

mentry, theme, tile

mentryWidget

Stichwörter: mentryWidget

Der Befehl `mentry::mentry`

Für Mentry Version 4.5

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Kurzreferenz
- Ausführliche Referenz

Kurzreferenz

NAME

`mentry::mentry` – Mehrfeld-Eingabe-Widgets erstellen und bearbeiten

ÜBERSICHT

```
mentry::mentry pathName ?options?
```

STANDARDOPTIONEN

```
-borderwidth          -highlightcolor      -relief
-highlightbackground  -highlightthickness
```

Optionen für alle Eingabe- und Beschriftungskomponenten

```
-background  -cursor  -font  -foreground
```

Optionen für alle Eingabekomponenten

```
-exportselection    -insertwidth        -selectforeground
-insertbackground   -invalidcommand     -show
-insertborderwidth  -justify            -state
-insertofftime      -selectbackground   -validate
-insertontime       -selectborderwidth  -validatecommand
```

WIDGET-SPEZIFISCHE OPTIONEN

```
-body {width ?text width text ...?}

-disabledbackground color

-disabledforeground color

-readonlybackground color

-takefocus 0|1|""|command

-textvariable array
```

INDIZES

```
number  end
```

WIDGET-BEFEHL

```
pathName adjustentry index string1 ?string2?

pathName attrib ?name? ?value name value ...?

pathName cget option

pathName clear first ?last?

pathName configure ?option? ?value option value ...?

pathName entries
```

```
  pathName entrycount

  pathName entrylimit index

  pathName entrypath index

  pathName getarray array

  pathName getlist

  pathName getstring

  pathName hasattrib name

  pathName isempty ?index?

  pathName isfull ?index?

  pathName labelcount

  pathName labelpath index

  pathName labels

  pathName put startIndex ?string string ...?

  pathName setentryextrawidth index amount

  pathName setentryfont index font

  pathName setentrywidth index width

  pathName unsetattrib name
```

SCHLÜSSELWÖRTER

mentry, widget, entry, label

Ausführliche Referenz

NAME

`mentry::mentry` – Mehrfeld-Eingabe-Widgets erstellen und bearbeiten

ÜBERSICHT

```
mentry::mentry pathName ?options?
```

BESCHREIBUNG

Der Befehl `mentry::mentry` erstellt ein neues Fenster mit dem Namen `pathName` und der Klasse `Mentry` und wandelt es in ein **mentry**-Widget um. Zusätzliche Optionen, die im

Folgenden beschrieben werden, können über die Befehlszeile oder in der Optionsdatenbank festgelegt werden, um Aspekte des mentry wie Farben, Schriftart und Komponenten zu konfigurieren. Der Befehl `mentry::mentry` gibt sein Argument `pathName` zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen `pathName` vorhanden sein, aber das übergeordnete Fenster von `pathName` muss vorhanden sein.

Das Wort *mentry* ist eine Abkürzung für *multi-entry* (Mehrfeld-Eingabe). Ein Mentry ist ein Megawidget, das aus einer beliebigen Anzahl von Einträgen besteht, die durch Beschriftungen voneinander getrennt und in einem Rahmen eingebettet sind. Durch entsprechend gewählte Konfigurationsoptionen für die Einträge und Beschriftungen sieht ein "Mentry"-Widget wie ein einziger Eintrag aus, der vorgefertigte Textabschnitte enthält, die vom Benutzer weder verschoben, geändert noch gelöscht werden können.

Im Mentry-Paket sind sowohl die Eingabekomponente als auch die Beschriftungskomponente direkte untergeordnete Elemente des Mentry-Widgets. Im Mentry_tile-Paket sind nur die Beschriftungen direkte Kinder, während die Eingabekomponenten Enkelkinder des "mentry"-Widgets sind (aus technischen Gründen sind die Eingabekomponenten eines auf Tile aufbauenden Mentry-Widgets in frame-Widgets eingebettet, die wiederum Kinder des Mentry-Widgets sind).

Die anfängliche Breite jeder Eingabekomponente (festgelegt durch die Konfigurationsoption `-body`) bestimmt auch die maximale Anzahl an Zeichen, die in diese eingefügt werden können. Dies wird mithilfe des Befehls `wcb::callback` erreicht, indem die Rückruffunktion `wcb::checkEntryLen` vor `insert` bei der Eingabekomponente registriert wird. Um diese Rückruffunktion beizubehalten, sollten zusätzliche before-`insert`-Rückruffunktionen (falls erforderlich) nur mit den Befehlen `wcb::cbappend` und `wcb::cbprepend` registriert werden. Wird diese Grenze bei einem Eintrag erreicht, der den Eingabefokus besitzt, wird dieser automatisch auf die nächste aktivierte Eingabekomponente gesetzt, der Inhalt dieses Widgets wird ausgewählt und der Einfügecursor wird an dessen Ende gesetzt. Dies wird durch die Definition einer entsprechenden Rückruffunktion nach `insert` sichergestellt, die als letzte solche Rückruffunktion für das Eingabefeld verbleiben muss. Aus diesem Grund sollten zusätzliche after-`insert`-Rückruffunktionen (falls erforderlich) nur mit dem Befehl `wcb::cbprepend` registriert werden. Derselbe Befehl sollte ausschließlich zur Definition von after-`motion`-Rückruffunktionen (falls erforderlich) verwendet werden, da `mentry::mentry` eine Rückruffunktion registriert, die für jede Eingabekomponente die letzte after-`motion` -Rückruffunktion sein muss.

Jedes Mentry-Widget kann eine beliebige Anzahl von **Attributen** aufweisen, die in Befehlen verwendet werden können, mit denen Mentry-Widgets für bestimmte Zwecke erstellt oder bearbeitet werden. Beispielsweise hat jedes vom Befehl `mentry::dateMentry` erzeugte Datums-Mentry ein Attribut `type` mit dem Wert `"Date"` und ein Attribut `format`, das die Bedeutungen seiner Komponenten festlegt. Die Werte dieser Attribute werden abgefragt und im Hauptteil der Prozeduren `mentry::putClockVal` und `mentry::getClockVal` verwendet.

STANDARDOPTIONEN

```
-borderwidth          -highlightcolor      -relief
-highlightbackground  -highlightthickness
```

Weitere Informationen zu den Standardoptionen für Tk-Widgets finden Sie im Handbucheintrag zu den **Optionen**. Diese Optionen werden nur vom Mentry-Paket unterstützt, nicht jedoch von Mentry_tile.

Optionen für alle Eingabe- und Beschriftungskomponenten

```
-background  -cursor  -font  -foreground
```

Diese Optionen (beschrieben im Handbucheintrag **options**) legen die Werte der gleichnamigen Optionen für alle Eingabekomponenten und Beschriftungskomponenten des angegebenen Mehrfeld-Eingabe-Widgets fest. Diese Werte können für eine einzelne Komponente mithilfe des Unterbefehls `configure` des dieser Komponente entsprechenden Tcl-Befehls geändert werden. Die Anwendung dieser Methode zur Änderung der Schriftart eines Eintrags in einem auf Tile aufbauenden "mentry"-Widget kann jedoch dessen Darstellung beeinträchtigen (dies hat technische Gründe, die mit Tile zusammenhängen). Um dieses Problem zu vermeiden, wird empfohlen, den Unterbefehl `setentryfont` des mit dem Mentry-Widget verknüpften Tcl-Befehls zu verwenden, um den Wert der Option `-font` einer seiner Eingabekomponenten zu ändern.

Wenn das Paket "Mentry_tile" verwendet wird, haben diese Optionen themenspezifische Standardwerte. Die gängigen Abkürzungen `-bg` (für `-background`) und `-fg` (für `-foreground`) werden sowohl für klassische als auch für Tile-basierte Mentry-Widgets unterstützt.

HINWEIS: Bitte beachten Sie, dass bei einigen Themen (wie `Arc`, `plastik`, `tileqt`, `vista` und `xpnative`) die Option `-background` für die Eingabekomponenten eines auf Tile aufbauenden Mentry-Widgets ignoriert wird. Das Gleiche gilt für das Thema `aqua` im Dunkelmodus sowie bei einigen Tk-Versionen auch im hellen Erscheinungsbild.

Optionen für alle Eingabekomponenten

```
-exportselection    -insertwidth        -selectforeground
-insertbackground   -invalidcommand     -show
-insertborderwidth  -justify            -state
-insertofftime      -selectbackground   -validate
-insertontime       -selectborderwidth  -validatecommand
```

Diese Optionen legen die Werte der gleichnamigen Optionen für alle Eingabekomponenten des angegebenen Mehrfeld-Eingabe-Widgets fest. Diese Werte können für eine einzelne Eingabekomponente mithilfe des Unterbefehls `configure` des diesem Eingabefeld entsprechenden Tcl-Befehls geändert werden. Die Optionen `-insertbackground`, `-insertborderwidth`, `-insertofftime`, `-insertontime`, `-insertwidth`, `-selectbackground`, `-selectborderwidth` und `-selectforeground` werden nur vom Mentry-Paket unterstützt, nicht jedoch von Mentry_tile. Die gängigen Abkürzungen `-invcmd` (für `-invalidcommand`) und `-vcmd` (für `-validatecommand`) werden sowohl für klassische als auch für auf Tile aufbauende Mentry-Widgets unterstützt.

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilenname:	-body
Name der Datenbank:	body
Datenbankklasse:	Body

Legt die Anfangsbreiten der Eingabekomponenten des angegebenen Mehrfeld-Eingabe-Widgets sowie die Texte fest, die in den Beschriftungen zwischen den Eingabefeldern angezeigt werden sollen. Der Wert dieser Option wird als Liste interpretiert. Jedes ungerade nummerierte Element wird als Wert der Konfigurationsoption `-width` einer Eingabekomponente interpretiert, während jedes gerade nummerierte Element den Wert der Konfigurationsoption `-text` eines Label-Unterelements darstellt. Das heißt, auf jede Eingabebreite muss ein Beschriftungstext folgen, mit Ausnahme der letzten, für die ein entsprechender Beschriftungstext optional ist. Die Reihenfolge der Listenelemente bestimmt die Erstellungsreihenfolge der Komponenten; diese werden in einer Zeile angeordnet, ohne horizontalen Innenabstand. Die anfängliche Breite jedes Eintrags bestimmt zudem die Anzahl der Zeichen, die in ihn eingefügt werden können; diese maximale Länge bleibt unverändert, selbst wenn die Breite des Eintrags später auf einen neuen Wert gesetzt wird.

Die Breite einer Eingabekomponente und der in einer Beschriftung angezeigte Text können mithilfe des Unterbefehls `configure` der dieser Komponente entsprechenden Tcl-Befehls geändert werden. Die Anwendung dieser Methode zur Änderung der Breite eines Eingabefelds eines auf Tile aufbauenden "mentry"-Widgets kann jedoch dessen Darstellung beeinträchtigen (dies hat technische Gründe, die mit der Tile-Technik zusammenhängen). Um dieses Problem zu vermeiden, wird empfohlen, den Unterbefehl `setentrywidth` des mit dem "mentry"-Widget verknüpften Tcl-Befehls zu verwenden, um den Wert der Option `-width` einer seiner Eingabekomponenten zu ändern.

Befehlszeilenname:	-disabledbackground
Name der Datenbank:	disabledBackground
Datenbankklasse:	DisabledBackground

Diese Option legt die Hintergrundfarbe fest, die für alle Eingabekomponenten und Beschriftungskomponenten des angegebenen Mehrfeld-Eingabe-Widgets verwendet werden soll, wenn dieses deaktiviert ist. Wenn der Wert dieser Option eine leere Zeichenkette ist, wird die normale Hintergrundfarbe verwendet. Diese Option wird nur vom Mentry-Paket unterstützt, nicht jedoch von Mentry_tile.

Befehlszeilenname:	-disabledforeground
Name der Datenbank:	disabledForeground
Datenbankklasse:	DisabledForeground

Diese Option legt die Vordergrundfarbe fest, die für alle Eingabekomponenten und

Beschriftungskomponenten des angegebenen Mehrfeld-Eingabe-Widgets verwendet werden soll, wenn dieses deaktiviert ist. Wenn der Wert dieser Option eine leere Zeichenkette ist, wird die normale Vordergrundfarbe verwendet. Diese Option wird nur vom Mentry-Paket unterstützt, nicht jedoch von Mentry_tile.

Befehlszeilenname:	-readonlybackground
Name der Datenbank:	readonlyBackground
Datenbankklasse:	ReadonlyBackground

Diese Option legt die Hintergrundfarbe fest, die für alle Eingabekomponenten und Beschriftungskomponenten des angegebenen Mehrfeld-Eingabe-Widgets verwendet werden soll, wenn dieses schreibgeschützt ist. Wenn der Wert dieser Option eine leere Zeichenkette ist, wird die normale Hintergrundfarbe verwendet. Diese Option wird nur vom Mentry-Paket unterstützt, nicht jedoch von Mentry_tile.

Befehlszeilenname:	-takefocus
Name der Datenbank:	takeFocus
Datenbankklasse:	TakeFocus

Diese Option legt fest, ob das Widget während der Tastaturnavigation den Fokus annimmt. Sie ist nahezu identisch mit der gleichnamigen Standardoption (Einzelheiten finden Sie im Handbucheintrag **options**). Der einzige Unterschied besteht darin, dass nicht das Widget selbst, sondern seine erste aktivierte Eingabekomponente den Fokus erhält, wenn die Standardtasten (`Tab` und `Shift-Tab`) bei der Tastaturnavigation gedrückt werden. Informationen dazu, wie Sie den Fokus mithilfe der Tastatur von einer Eingabekomponente zu einer anderen verschieben können, finden Sie im Abschnitt VORGABEBINDUNGEN weiter unten.

Befehlszeilenname:	-textvariable
Name der Datenbank:	textvariable
Datenbankklasse:	Variable

Wenn der Wert dieser Option keine leere Zeichenkette ist, muss es sich um ein Array handeln. In diesem Fall legt für jede Eingabekomponente des angegebenen Mehrfeld-Eingabe-Widgets der Wert des Array-Elements mit dem Index der Eingabe als Elementname den Wert der Option `-textvariable` für diese Eingabe fest. Die Eintragsindizes beginnen bei `0`. Beispielsweise entspricht die Angabe von `-textvariable arr` für ein Mehrfeld-Eingabe-Widget mit zwei Eingabekomponenten der Angabe von `-textvariable arr(0)` für den ersten und `-textvariable arr(1)` für den zweiten Eintrag dieses Widgets.

INDIZES

Einige der Widget-Befehle für Mentries nehmen einen Index als Argument entgegen. Ein Index legt einen bestimmten Eintrag oder eine bestimmte Beschriftungskomponente des Mentry

fest auf eine der folgenden Weisen:

number	Legt die Komponente als numerischen Index fest, wobei 0 die erste Eingabe-oder Beschriftungskomponente bezeichnet.
end	Bezeichnet die letzte Eingabe-oder Beschriftungskomponente des Mentry.

Das Wort `end` kann als `en` oder `e` abgekürzt werden. Indizes, die außerhalb des zulässigen Bereichs liegen, werden automatisch auf den nächstgelegenen zulässigen Wert gerundet.

WIDGET-BEFEHL

Der Befehl `mentry::mentry` erstellt einen neuen Tcl-Befehl mit dem Namen `pathName`. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget auszuführen. Er hat die folgende allgemeine Form:

```
pathName option ?arg arg ...?
```

`option` und `arg` legen das genaue Verhalten des Befehls fest. Für Mentry-Widgets sind folgende Befehle möglich:

`pathName adjustentry index string1 ?string2?`

Erhöht bei Bedarf die Breite der durch `index` identifizierten Eingabekomponente in Pixeln, sodass sie gerade groß genug für Texte der eingabespezifischen maximalen Länge ist, die aus den im Argument `string1` enthaltenen Zeichen bestehen, mit Ausnahme von höchstens einem Zeichen, das im optionalen Argument `string2` enthalten sein darf. Gibt eine leere Zeichenkette zurück.

Wenn Sie beispielsweise ein Eingabe-Widget erstellen, dessen Komponenten ausschließlich bis zu zwei hexadezimale Ziffern in Großbuchstaben akzeptieren, können Sie durch Übergabe der Zeichenfolge `"0123456789ABCDEF"` an diesen Unterbefehl sicherstellen, dass die Eingabekomponenten eine optimale Breite haben, gerade groß genug, um nicht nur die Dezimalziffern, sondern auch bis zu zwei Buchstaben im Bereich `A` – `F` anzuzeigen (die bei Verwendung einer proportionalen Schriftart mehr Platz benötigen als die Dezimalziffern). Es wird empfohlen, diesen Unterbefehl auch dann aufzurufen, wenn die zulässigen Zeichen ausschließlich auf die Ziffern `0` – `9` beschränkt sind, da bei einigen Schriftarten nicht alle Dezimalziffern die gleiche Breite haben.

Ein Beispiel für den Fall, in dem beide Zeichenfolgenargumente verwendet werden, ist ein Mentry-Widget für (vorzeichenbehaftete) reelle Zahlen im Festkommaformat. Für seine erste Eingabekomponente (die den ganzzahligen Teil einer reellen Zahl anzeigt) sollte dieser Unterbefehl mit `"0123456789"` als Argument `string1` und `"+-"` als Argument `string2` aufgerufen werden, da das erste Zeichen des Eintrags nicht nur eine Dezimalziffer, sondern auch das Vorzeichen `"+"` oder `"-"` sein kann. (HINWEIS: Bei den meisten proportionalen Schriftarten ist die Breite des Zeichens `"+"` größer als die von `"0"`.)

`pathName attrib ?name? ?value name value ...?`

Fragt die Attribute des Widgets ab oder ändert sie. Ist kein `name` angegeben, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für

`pathName` enthält. Ist `name` ohne `value` angegeben, gibt der Befehl den Wert des genannten Attributs zurück, oder eine leere Zeichenkette, wenn kein entsprechender Wert existiert (mit dem Unterbefehl `hasattrib` unterscheiden Sie diesen Fall von dem, in dem der Wert eines *vorhandenen* Attributs eine leere Zeichenkette ist). Sind ein oder mehrere Paare aus `name` und `value` angegeben, setzt der Befehl die genannten Widget-Attribute auf die genannten Werte; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. `name` kann eine beliebige Zeichenkette sein.

`pathName cget option`
Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption zurück, die einen der vom Befehl `mentry::mentry` akzeptierten Werte annehmen kann.

`pathName clear first ?last?`
Löscht den Inhalt einer oder mehrerer Eingabekomponenten des Widgets. `first` und `last` sind Indizes, die den ersten und letzten Eintrag des zu löschenden Bereichs festlegen. Wenn `last` nicht festgelegt wird, wird standardmäßig `first` verwendet, d. h., ein einzelner Eintrag wird gelöscht. Der Wert ist eine leere Zeichenkette.

`pathName configure ?option? ?value option value ...?`
Ruft die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn kein `option` angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für `pathName` beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des zurückgegebenen Werts, wenn kein `option` festgelegt wurde). Wenn ein oder mehrere Paare aus `option` und `value` angegeben werden, ändert der Befehl die angegebenen Widget-Optionen so, dass sie die angegebenen Werte annehmen; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. `option` kann jeden der vom Befehl `mentry::mentry` akzeptierten Werte annehmen.

`pathName entries`
Gibt eine Liste zurück, die die Pfadnamen aller Eingabekomponenten des Widgets enthält.

`pathName entrycount`
Gibt die Anzahl aller Eingabekomponenten des Widgets zurück.

`pathName entrylimit index`
Gibt die Anzahl der Zeichen zurück, die in die durch `index` identifizierte Eingabekomponente eingefügt werden können.

`pathName entrypath index`
Gibt den Pfadnamen der durch `index` identifizierten Eingabekomponente zurück.

`pathName getarray array`
Für jede Eingabekomponente des Widgets weist der Befehl die in der Eingabe enthaltene Zeichenkette dem Element von `array` zu, dessen Index dem Elementnamen der Eingabe entspricht. Die Indizes der Eingaben beginnen bei `0`. Der Rückgabewert ist eine leere Zeichenkette.

`pathName getlist`
Gibt eine Liste zurück, die die Zeichenfolgen aller Eingabekomponenten des Widgets enthält.

`pathName getstring`
Gibt eine Zeichenkette zurück, die durch Verknüpfung der in den Eingabekomponenten des Widgets enthaltenen Zeichenketten mit den Texten gebildet wird, die in den die Eingaben

trennenden Beschriftungen angezeigt werden, und zwar in der Reihenfolge der Erstellung der Komponenten, wie sie durch die Konfigurationsoption `-body` vorgegeben ist. Das heißt, auf die Zeichenkette des ersten Eintrags folgt der Text der ersten Beschriftung, auf den wiederum die Zeichenkette des zweiten Eintrags folgt, und so weiter.

`pathName hasattrib name`
Gibt `1` zurück, wenn das Attribut `name` vorhanden ist, andernfalls `0`.

`pathName isempty ?index?`
Wenn `index` festgelegt wird, gibt der Befehl den Wert `1` zurück, wenn die Zeichenfolge in der durch `index` identifizierten Eingabekomponente leer ist, und andernfalls den Wert `0`. Fehlt dieses optionale Argument, gibt der Befehl den Wert `1` zurück, wenn die in den Eingabekomponenten des Widgets enthaltenen Zeichenfolgen alle leer sind, und andernfalls den Wert `0`.

`pathName isfull ?index?`
Wenn `index` festgelegt wird, gibt der Befehl den Wert `1` zurück, wenn die Länge der Zeichenkette, die in der durch `index` identifizierten Eingabekomponente enthalten ist, der Anzahl der Zeichen entspricht, die in den Eintrag eingefügt werden können, und andernfalls den Wert `0`. Ist dieses optionale Argument nicht vorhanden, gibt der Befehl den Wert `1` zurück, wenn die Längen der Zeichenfolgen in allen Eingabekomponenten des Widgets der Anzahl der Zeichen entsprechen, die in die jeweiligen Einträge eingefügt werden können (d. h., wenn alle Eingabekomponenten voll sind) und andernfalls den Wert `0`.

`pathName labelcount`
Gibt die Anzahl aller untergeordneten Label-Elemente des Widgets zurück.

`pathName labelpath index`
Gibt den Pfadnamen des durch `index` identifizierten untergeordneten Labels zurück.

`pathName labels`
Gibt eine Liste zurück, die die Pfadnamen aller untergeordneten Label-Elemente des Widgets enthält.

`pathName put startIndex ?string string ...?`
Ersetzt die Texte der Eingabekomponenten, deren Indizes größer oder gleich `startIndex` sind, durch die entsprechenden `string`-Argumente unter Verwendung der Operationen `delete` und `insert`, bis entweder die Einträge oder die `string`s aufgebraucht sind. Wird einer dieser Unterbefehle durch einen before-Rückruffunktion in einer beliebigen Eingabekomponente abgebrochen, stellt der Befehl den ursprünglichen Inhalt aller betroffenen Einträge wieder her und gibt den Wert `0` zurück; andernfalls gibt er `1` zurück. In beiden Fällen behält der Befehl die Position des Einfügecursors in allen Eingabekomponenten bei.

`pathName setentryextrawidth index amount`
Legt eine zusätzliche Breite für die durch `index` identifizierte Eingabekomponente fest. `amount` kann eine beliebige der Standardformen für Bildschirmabstände annehmen (z. B. eine Pixelanzahl). Der Rückgabewert ist eine leere Zeichenkette.

Beachten Sie, dass eine Änderung der Breite einer Eingabekomponente keinen Einfluss auf die Anzahl der Zeichen hat, die in diese eingegeben werden können; diese bleibt auf den Wert der Option `-body` der entsprechenden Komponente festgelegt.

`pathName setentryfont index font`

Setzt die Schriftart der durch `index` identifizierten Eingabekomponente auf den durch `font` angegebenen Wert. Dies geschieht durch Aufruf des Unterbefehls `configure` des dem Eintrag entsprechenden Tcl-Befehls. Bei Verwendung des Pakets Mentry_tile führt dieser Befehl außerdem einige zusätzliche Schritte durch, die zur Aktualisierung des Erscheinungsbilds des auf Tile aufbauenden Mentry-Widgets erforderlich sind. Der Rückgabewert ist eine leere Zeichenkette.

`pathName setentrywidth index width`

Stellt die Breite der durch `index` identifizierten Eingabekomponente auf die durch `width` angegebene Zeichenanzahl ein. Dies geschieht durch Aufruf des Unterbefehls `configure` des dem Eintrag entsprechenden Tcl-Befehls. Bei Verwendung des Pakets Mentry_tile führt dieser Befehl außerdem einige zusätzliche Schritte aus, die zur Aktualisierung der Darstellung des auf Tile aufbauenden Mentry-Widgets erforderlich sind. Der Rückgabewert ist eine leere Zeichenkette.

Beachten Sie, dass eine Änderung der Breite einer Eingabekomponente keinen Einfluss auf die Anzahl der Zeichen hat, die in diese eingegeben werden können; diese bleibt auf den Wert der Option `-body` der entsprechenden Komponente festgelegt.

`pathName unsetattrib name`

Hebt das Attribut `name` auf. Gibt eine leere Zeichenfolge zurück.

VORGABEBINDUNGEN

Der Befehl `mentry::mentry` definiert die Bindungen der Komponenten des von ihm erstellten "mentry"-Widgets teilweise neu:

1. Durch Klicken mit der linken Maustaste in ein untergeordnetes Feld einer Beschriftung wird der Einfügecursor an das Ende des letzten aktivierten Eintrags vor der Beschriftung gesetzt, der Fokus auf diesen Eintrag gelegt und die Auswahl dort gelöscht.
2. Wenn in einer nicht leeren Eingabekomponente ein Zeichen eingegeben wird, das in der diesem Eingabekomponente folgenden Beschriftung (sofern vorhanden) enthalten ist, wird der Fokus auf die nächste aktivierte Eingabekomponente verschoben, der Inhalt dieser Eingabekomponente wird ausgewählt und der Einfügecursor wird an dessen Ende gesetzt.
3. Am Anfang einer Eingabekomponente, die nicht die erste aktivierte Eingabekomponente innerhalb des Widgets ist, löscht die Taste `Left` die Auswahl in dieser Eingabekomponente, verschiebt den Fokus auf die vorherige aktivierte Eingabekomponente und setzt den Einfügecursor an das Ende dieses Widgets. Ebenso löscht die Taste `Right` am Ende einer Eingabekomponente, die nicht das letzte aktivierte Eingabefeld innerhalb des Widgets ist, die Auswahl in diesem Eingabefeld, verschiebt den Fokus auf die nächste aktivierte Eingabekomponente und setzt den Einfügecursor an den Anfang dieses Widgets. Wenn `tk_strictMotif` auf "false" gesetzt ist, verhalten sich `Control-b` und `Control-f` jeweils genauso wie `Left` und `Right`.
4. In einer Eingabekomponente, die nicht die erste aktivierte Eingabekomponente innerhalb des Widgets ist, verschiebt `Control-Left` den Fokus auf die vorherige aktivierte Eingabekomponente, wählt den Inhalt dieses Widgets aus und setzt den Einfügecursor an dessen Ende; gibt es links von der aktuellen Eingabekomponente keine aktivierte Eingabekomponente, so verschiebt `Control-Left` den Einfügecursor an den Anfang des aktuellen Eintrags und löscht die Auswahl in diesem Widget. Ähnlich verhält es sich bei einer Eingabekomponente, die nicht der letzte aktivierte Eintrag innerhalb des Widgets ist:

`Control-Right` verschiebt den Fokus auf die nächste aktivierte Eingabekomponente, wählt den Inhalt dieses Widgets aus und setzt den Einfügecursor an dessen Ende; befindet sich rechts vom aktuellen Eintrag keine aktivierte Eingabekomponente, so bewegt `Control-Right` den Einfügecursor an das Ende des aktuellen Eintrags und löscht die Auswahl in diesem Widget. Wenn `tk_strictMotif` falsch ist, verhalten sich `Meta-b` und `Meta-f` jeweils genauso wie `Control-Left` und `Control-Right`.

5. Die Taste `Home` löscht die Auswahl im aktuellen Eingabefeld, verschiebt den Fokus auf die erste aktivierte Eingabekomponente und setzt den Einfügecursor an den Anfang dieses Widgets. Ebenso löscht die Taste `End` die Auswahl im aktuellen Eingabe-Widget, verschiebt den Fokus auf die letzte aktivierte Eingabekomponente und setzt den Einfügecursor an das Ende dieses Widgets. Wenn `tk_strictMotif` falsch ist, verhalten sich `Control-a` und `Control-e` jeweils genauso wie `Home` und `End`.
6. `Shift-Home` verschiebt den Fokus auf die erste aktivierte Eingabekomponente, setzt den Einfügecursor an den Anfang dieses Widgets und erweitert entweder die Auswahl bis zu dieser Position oder löscht die Auswahl im aktuellen Eintrag und wählt den Inhalt des neuen Widgets aus, je nachdem, ob der aktuelle Eintrag die erste aktivierte Eingabekomponente ist oder nicht. In ähnlicher Weise verschiebt `Shift-End` den Fokus auf die letzte aktivierte Eingabekomponente, setzt den Einfügecursor an das Ende dieses Widgets, und erweitert entweder die Auswahl bis zu dieser Position oder löscht die Auswahl im aktuellen Eintrag und markiert den Inhalt des neuen Widgets, je nachdem, ob der aktuelle Eintrag die letzte aktivierte Eingabekomponente ist oder nicht.
7. Die Taste `BackSpace` löscht die Auswahl, wenn sie im aktuellen Eintrag vorhanden ist. Andernfalls löscht sie entweder das Zeichen links vom Einfügecursor im aktuellen Eintrag oder das letzte Zeichen der vorherigen aktivierten Eingabekomponente. Letzteres geschieht am Anfang einer Eingabekomponente, die nicht der erste aktivierte Eintrag innerhalb des Widgets ist; in diesem Fall verschiebt die Taste `BackSpace` zudem den Fokus auf die vorherige aktivierte Eingabekomponente und setzt den Einfügecursor an deren Ende. Wenn `tk_strictMotif` auf "false" gesetzt ist, verhält sich `Control-h` genauso wie `BackSpace`.
8. Wenn `tk_strictMotif` auf "false" gesetzt ist, löscht `Meta-d` alle Zeichen rechts vom Einfügecursor innerhalb des aktuellen Eintrags, während `Meta-BackSpace` und `Meta-Delete` entweder alle Zeichen links vom Einfüge-Cursor im aktuellen Eintrag oder den Inhalt der vorherigen aktivierten Eingabekomponente löschen. Letzteres geschieht am Anfang einer Eingabekomponente, die nicht der erste aktivierte Eintrag innerhalb des Widgets ist; in diesem Fall löschen `Meta-BackSpace` und `Meta-Delete` zudem die Auswahl im aktuellen Eintrags-Widget und verschieben den Fokus auf die vorherige aktivierte Eingabekomponente.

SCHLÜSSELWÖRTER

mentry, widget, entry, label

wcbRef

Stichwörter: wcbRef

Wcb-Befehlsreferenz

Für Wcb Version 4.2

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Der Befehl wcb::callback
- Der Befehl wcb::cbappend
- Der Befehl wcb::cbprepend
- Der Befehl wcb::cancel
- Der Befehl wcb::canceled
- Der Befehl wcb::extend
- Der Befehl wcb::replace
- Der Befehl wcb::pathname
- Der Befehl wcb::changeEntryText
- Der Befehl wcb::postInsertEntryLen
- Der Befehl wcb::postInsertEntryText
- Der Befehl wcb::postDeleteEntryText
- before-insert-Rückruffunktionen für entry-, spinbox- und combobox-Widgets
- before-insert-Rückruffunktionen für text- und ctext-Widgets

---

Der Befehl `wcb::callback`

NAME

`wcb::callback` – Widget-Rückruffunktionen abfragen, setzen und entfernen

ÜBERSICHT

```
wcb::callback widgetName before|after option ?callback callback ...?
```

BESCHREIBUNG

Fragt die Rückruffunktionen für das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox-, Ttk-combobox-, listbox-, tablelist-, Ttk-treeview-, text- oder ctext-Widget `widgetName`, das Argument `before` oder `after` und den Befehl zu `option` ab, setzt sie oder entfernt sie. Die Werte des Arguments `option` können sein:

- `insert`, `delete` oder `motion`, für ein Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox-, Ttk-combobox-, text- oder ctext-Widget;
- `replace`, für ein text- oder ctext-Widget;
- `activate`, für ein listbox-, tablelist- oder Ttk-treeview-Widget;
- `selset` oder `selclear`, für ein listbox-, tablelist-, Ttk-treeview-, text- oder ctext-Widget;
- `seladd` oder `seltoggle`, für ein Ttk-treeview-Widget;
- `cellselset` oder `cellselclear`, für ein tablelist- oder Ttk-treeview-Widget;
- `cellseladd` oder `cellseltoggle`, für ein Ttk-treeview-Widget;
- `activatecell`, für ein tablelist-Widget.

Ist nach dem Parameter `option` kein Argument angegeben, gibt die Prozedur nur die aktuelle before- bzw. after-Rückruffunktionsliste für die gegebene Widget-Operation zurück.

Andernfalls:

- Ist mindestens eines der Argumente nach dem Parameter `option` eine nichtleere Zeichenkette, dann:
- Sind alle Argumente nach dem Parameter `option` leer, hebt die Prozedur die Eintragung aller zugehörigen before- oder after-Rückruffunktionen für das gegebene Widget auf und gibt eine leere Zeichenkette zurück.
  - wird der Befehl für dieses Widget zum ersten Mal mit mindestens einem nichtleeren Argument nach dem Parameter `option` aufgerufen, benennt die Prozedur den Tcl-Befehl `::widgetName` in `::_widgetName` um und baut eine neue Prozedur `::widgetName`, in der die Ausführung der Widget-Operationen zu den oben genannten Werten von `option` von Aufrufen der zugehörigen before-Rückruffunktionen eingeleitet und von Aufrufen der zugehörigen after-Rückruffunktionen im globalen Geltungsbereich gefolgt wird;
  - sie setzt die Rückruffunktionsliste auf die aus diesen Argumenten gebildete Liste und gibt die neue Liste zurück.

Wird eine Rückruffunktion für das Widget `widgetName` aufgerufen, werden der Name des **Proxy-Befehls** `::_widgetName` und die Befehlsargumente automatisch als Parameter angehängt. Mit dem Befehl `wcb::pathname` holen Sie den ursprünglichen Widget-Pfadnamen.

Die folgende Tabelle zeigt die Widget-Unterbefehle zu den oben genannten Werten von `option`, zusammen mit den Argumenten dieser Unterbefehle:

Widget	**option**	**Unterbefehl**	**Argumente**
Tk entry, ttk::entry, BWidget Entry, Tk spinbox, ttk::spinbox, oder ttk::combobox	insert	insert	index string
delete	delete	from ?to?	
motion	icursor	index	
listbox	activate	activate	index
selset	selection set	first ?last?	
selclear	selection clear	first ?last?	
tablelist::tablelist	activate	activate	index
selset	selection set	first ?last? (indexList oder firstIndex lastIndex)	
selclear	selection clear	first ?last? (indexList oder firstIndex lastIndex)	
activatecell	activatecell	cellIndex	

Widget	option	Unterbefehl	Argumente
cellselset	cellselection set	first ?last? (cellIndexList oder firstCell lastCell)	
cellselclear	cellselection clear	first ?last? (cellIndexList oder firstCell lastCell)	
ttk::treeview	activate	focus	item
selset	selection set	itemList	
seladd	selection add	itemList	
selclear	selection remove	itemList	
seltoggle	selection toggle	itemList	
cellselset	cellselection set	first ?last? (cellList oder firstCell lastCell)	
cellseladd	cellselection add	first ?last? (cellList oder firstCell lastCell)	
cellselclear	cellselection remove	first ?last? (cellList oder firstCell lastCell)	
cellseltoggle	cellselection toggle	first ?last? (cellList oder firstCell lastCell)	
text oder ctext	insert	insert	index string ?tagList string tagList ...?
delete	delete	from ?to?	
replace	replace	from to string ?tagList string tagList ...?	
motion	mark set insert	index	
selset	tag add sel	from ?to from to ...?	
selclear	tag remove sel	from ?to from to ...?	

HINWEISE:

1. Die Wörter `before`, `after`, `insert`, `delete`, `replace`, `motion` und `activate` (letzteres nicht bei tablelist-Widgets) dürfen Sie auf mindestens ein Zeichen kürzen. Ebenso genügen die ersten vier Zeichen der Wörter `selset`, `selclear`, `seladd` und `seltoggle`, die ersten acht Zeichen von `cellselset`, `cellselclear`, `cellseladd` und `cellseltoggle` und die ersten neun Zeichen von `activatecell`, damit Wcb diese Optionen erkennt.

2. Nach einem erfolgreichen Aufruf dieses Befehls mit mindestens einer nichtleeren Rückruffunktion nach dem Argument `option` können Sie entweder die neue Prozedur `widgetName` oder den Proxy-Befehl `::_widgetName` verwenden, um jede gültige Operation auf dem Widget `widgetName` auszuführen. Verwenden Sie den Proxy-Befehl `::_widgetName`, wenn die Rückruffunktionen beim Ausführen des jeweiligen Widget-Unterbefehls nicht aufgerufen werden sollen.
3. Wird ein Widget `widgetName` zerstört, für das `wcb::callback` den zugehörigen Tcl-Befehl durch eine neue Prozedur ersetzt hat, löscht der Tcl-Interpreter den Proxy-Befehl `::_widgetName` automatisch. Das gilt nicht für ein BWidget Entry, ein tablelist- oder ein ctext-Widget. Die neue Widget-Prozedur `widgetName` bliebe bestehen, Wcb sorgt aber dafür, dass sie aus einem Aufräumskript gelöscht wird, das an das Ereignis `<Destroy>` gebunden ist. Dieses Aufräumskript gehört zu einem Bindungs-Tag namens `WcbCleanup`, das der Liste der Bindungs-Tags des Widgets angehängt wird, wenn zum ersten Mal Rückruffunktionen dafür eingetragen werden. Bei einem BWidget Entry, einem tablelist- oder einem ctext-Widget löscht dieses Skript auch den Proxy-Befehl `::_widgetName`.
4. Das genannte Aufräumskript hebt außerdem die Eintragung aller für `widgetName` festgelegten Rückruffunktionen auf. Ein später mit demselben Pfad erzeugtes Widget erbt sie dadurch nicht von dem gerade gelöschten Widget. Das kann in manchen Anwendungen wichtig sein. Entfernen Sie `WcbCleanup` deshalb nicht aus der Liste der Bindungs-Tags des gegebenen Widgets.

SCHLÜSSELWÖRTER

callback, widget, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, listbox, tablelist, Ttk treeview, text, ctext

---

Der Befehl `wcb::cbappend`

NAME

`wcb::cbappend` – An eine Rückruffunktionsliste anhängen

ÜBERSICHT

```
wcb::cbappend widgetName before|after option ?callback callback ...?
```

BESCHREIBUNG

Dieser Befehl ist nahezu identisch mit `wcb::callback`. Der einzige Unterschied: `wcb::cbappend` *hängt* die nach dem Parameter `option` angegebenen Argumente an die aktuelle Rückruffunktionsliste an, falls eine vorhanden ist, während `wcb::callback` die alten Rückruffunktionen durch diese Argumente *ersetzt*.

SCHLÜSSELWÖRTER

callback, append, widget

Der Befehl `wcb::cbprepend`

NAME

`wcb::cbprepend` – Einer Rückruffunktionsliste voranstellen

ÜBERSICHT

```
wcb::cbprepend widgetName before|after option ?callback callback ...?
```

BESCHREIBUNG

Dieser Befehl ist nahezu identisch mit `wcb::callback`. Der einzige Unterschied: `wcb::cbprepend` *stellt* die nach dem Parameter `option` angegebenen Argumente der aktuellen Rückruffunktionsliste *voran*, falls eine vorhanden ist, während `wcb::callback` die alten Rückruffunktionen durch diese Argumente *ersetzt*.

SCHLÜSSELWÖRTER

callback, prepend, widget

Der Befehl `wcb::cancel`

NAME

`wcb::cancel` – Einen Widget-Befehl abbrechen

ÜBERSICHT

```
wcb::cancel ?script?
```

BESCHREIBUNG

Diese Prozedur ist dafür gedacht, aus einer before-Rückruffunktion für einen Widget-Befehl aufgerufen zu werden. Sie bricht die Ausführung dieses Befehls und der übrigen Rückruffunktionen ab und wertet das Argument `script` im globalen Geltungsbereich aus. Ist dieses Argument nicht angegeben, ist die Vorgabe der Befehl `bell`.

Der Rückgabewert ist der von `script`, wenn dieses Argument angegeben und nichtleer ist. Sonst gibt der Befehl eine leere Zeichenkette zurück.

SCHLÜSSELWÖRTER

cancel, command, widget

Der Befehl `wcb::canceled`

NAME

`wcb::canceled` – Den Abbruchzustand eines Widget-Befehls abfragen

ÜBERSICHT

```
wcb::canceled widgetName option
```

BESCHREIBUNG

Gibt den Wert `1` zurück, wenn der jüngste Aufruf der Widget-Operation zu `widgetName` und `option` von einer before-Rückruffunktion durch den Aufruf von `wcb::cancel` abgebrochen wurde; sonst ist der Rückgabewert `0`. Die Argumente müssen dieselben Einschränkungen erfüllen wie beim Befehl `wcb::callback`.

SCHLÜSSELWÖRTER

cancel, command, widget

Der Befehl `wcb::extend`

NAME

`wcb::extend` – Die Argumentliste eines Widget-Befehls erweitern

ÜBERSICHT

```
wcb::extend ?arg arg ...?
```

BESCHREIBUNG

Diese Prozedur ist dafür gedacht, aus einer before-Rückruffunktion für einen Widget-Befehl aufgerufen zu werden. Sie hängt die in den optionalen Parametern `arg` gegebenen Werte an die Argumentliste dieses Befehls an. Die neue Argumentliste wird auch an die übrigen Rückruffunktionen für diesen Befehl weitergegeben.

Diese Prozedur gibt ihre Parameter einfach an den Befehl `lappend` weiter, aufgerufen für die Argumentliste der jeweiligen Widget-Operation.

SCHLÜSSELWÖRTER

extend, argument, command, widget

Der Befehl `wcb::replace`

NAME

`wcb::replace` – Argumente eines Widget-Befehls durch neue ersetzen

ÜBERSICHT

```
wcb::replace first last ?arg arg ...?
```

BESCHREIBUNG

Diese Prozedur ist dafür gedacht, aus einer before-Rückruffunktion für einen Widget-Befehl aufgerufen zu werden. Sie ersetzt die Argumente mit den Indizes `first` bis `last` dieses Befehls durch die optionalen Parameter `arg`. Die neue Argumentliste wird auch an die übrigen Rückruffunktionen für diesen Befehl weitergegeben. Die Argumente sind ab `0` nummeriert (siehe die Tabelle in der Beschreibung des Befehls `wcb::callback`).

Diese Prozedur gibt ihre Parameter einfach an den Befehl `lreplace` weiter, aufgerufen für die Argumentliste der jeweiligen Widget-Operation.

SCHLÜSSELWÖRTER

replace, argument, command, widget

Der Befehl `wcb::pathname`

NAME

`wcb::pathname` – Den Pfadnamen des Widgets zu einem Tcl-Befehlsnamen abfragen

ÜBERSICHT

```
wcb::pathname proxyCmd
```

BESCHREIBUNG

Diese Prozedur gibt den Pfadnamen des Widgets zurück, das zu dem Tcl-Befehlsnamen `proxyCmd` gehört, der an eine Rückruffunktion übergeben wurde.

Wird eine before- oder after-Rückruffunktion für ein Widget aufgerufen, wird der Name des zugehörigen Proxy-Befehls automatisch als Parameter angehängt. Innerhalb einer Rückruffunktion holt diese Prozedur den Pfadnamen des Widgets aus dem Befehlsnamen, der der Rückruffunktion als Argument übergeben wurde. Sie gibt einfach den Zeichenkettenbereich zurück, der aus `proxyCmd` durch Entfernen des Präfixes `"::_"` entsteht.

SCHLÜSSELWÖRTER

path name, command, widget

Der Befehl `wcb::changeEntryText`

NAME

`wcb::changeEntryText` – Den Text eines Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widgets ändern

ÜBERSICHT

```
wcb::changeEntryText widgetName string
```

BESCHREIBUNG

Ersetzt den Text des Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widgets `widgetName` durch `string`, über die Operationen `delete` und `insert`. Bricht eine before-`delete`-Rückruffunktion den ersten Unterbefehl ab, kehrt die Prozedur zurück, ohne den neuen Text einzufügen. Bricht eine before-`insert`-Rückruffunktion die zweite Operation ab, stellt der Befehl den ursprünglichen Inhalt des Widgets wieder her.

Die Prozedur behält die Position der Einfügemarke bei. Der Rückgabewert ist `1` bei Erfolg und `0` bei Fehlschlag, also wenn eine der versuchten Operationen von einer before-Rückruffunktion abgebrochen wurde.

SCHLÜSSELWÖRTER

Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Der Befehl `wcb::postInsertEntryLen`

NAME

`wcb::postInsertEntryLen` – Die Länge abfragen, die der Text in einem Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget nach dem Einfügen hätte

ÜBERSICHT

```
wcb::postInsertEntryLen widgetName string
```

BESCHREIBUNG

Gibt die Länge des Textes zurück, den das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `widgetName` nach dem Einfügen von `string` enthielte.

SCHLÜSSELWÖRTER

insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Der Befehl `wcb::postInsertEntryText`

NAME

`wcb::postInsertEntryText` – Den Text abfragen, den ein Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget nach dem Einfügen hätte

ÜBERSICHT

```
wcb::postInsertEntryText widgetName index string
```

BESCHREIBUNG

Gibt den Text zurück, den das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `widgetName` enthielte, nachdem `string` vor dem Zeichen eingefügt wurde, das `index` bezeichnet.

SCHLÜSSELWÖRTER

insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Der Befehl `wcb::postDeleteEntryText`

NAME

`wcb::postDeleteEntryText` – Den Text abfragen, den ein Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget nach dem Löschen hätte

ÜBERSICHT

```
wcb::postDeleteEntryText widgetName from ?to?
```

BESCHREIBUNG

Gibt den Text zurück, den das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `widgetName` enthielte, nachdem die Zeichen ab dem von `from` bezeichneten und unmittelbar vor `to` gelöscht wurden. Ist `to` nicht angegeben, ist der Rückgabewert der Text, den das Widget nach dem Löschen des einzelnen von `from` gegebenen Zeichens enthielte.

HINWEIS: Dieser Befehl hat eine veränderliche Anzahl von Argumenten (2 oder 3), weil der Unterbefehl `delete` des zu `widgetName` gehörenden Tcl-Befehls entweder einen oder zwei Indizes als Argumente erwartet. Der richtige Aufruf aus einer before-`delete`-Rückruffunktion sieht deshalb so aus wie im folgenden Beispiel:

```
proc myBeforeDeleteCallback {w args} {
    #
    # Get the text that would be contained in the widget after
    # deleting the characters specified by $args, which stands
    # for the one or two arguments passed to delete; pass these
    # arguments to wcb::postDeleteEntryText by expanding $args
    #
    set newText [eval [list wcb::postDeleteEntryText $w] $args]
```

```
    if {!some_condition_on_$newText} {
        wcb::cancel
    }
}
```

Die folgende alternative, elegantere Lösung braucht Tcl/Tk 8.5 oder neuer:

```
proc myBeforeDeleteCallback {w args} {
    # . . .
    set newText [wcb::postDeleteEntryText $w {*}$args]
    . . .
}
```

SCHLÜSSELWÖRTER

delete, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

before-insert-Rückruffunktionen für entry-, spinbox- und combobox-Widgets

NAME

`wcb::checkStrFor*`, `wcb::convStrTo*`, `wcb::checkEntryFor*`, `wcb::checkEntryLen` – before-`insert`-Rückruffunktionen für Tk-entry-, Ttk-entry-, BWidget-Entry-, Tk-spinbox-, Ttk-spinbox- und Ttk-combobox-Widgets

ÜBERSICHT

```
wcb::checkStrForRegExp  exp w idx str

wcb::checkStrForAlpha       w idx str
wcb::checkStrForNum         w idx str
wcb::checkStrForAlnum       w idx str

wcb::convStrToUpper         w idx str
wcb::convStrToLower         w idx str

wcb::checkEntryForInt       w idx str
wcb::checkEntryForUInt  max w idx str
wcb::checkEntryForReal      w idx str
wcb::checkEntryForFixed cnt w idx str

wcb::checkEntryLen      len w idx str
```

BESCHREIBUNG

Die Rückruffunktion `wcb::checkStrForRegExp` prüft, ob der reguläre Ausdruck `exp` die Zeichenkette `str` trifft, die in das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` eingefügt werden soll. Trifft er sie nicht, bricht sie die Operation `insert` ab.

Die drei anderen Rückruffunktionen `wcb::checkStrFor*` prüfen, ob die Zeichenkette `str`,

die in das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` eingefügt werden soll, alphabetisch, numerisch bzw. alphanumerisch ist. Ist sie es nicht, brechen sie die Operation `insert` ab. Diese Prozeduren rufen nur die Rückruffunktion `wcb::checkStrForRegExp` auf und übergeben ihr die Unicode-Muster `{^[[:alpha:]]*$}`, `{^[[:digit:]]*$}` und `{^[[:alnum:]]*$}`.

Die Rückruffunktionen `wcb::convStrTo*` ersetzen die Zeichenkette `str`, die in das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` eingefügt werden soll, durch ihre Entsprechung in Groß- bzw. Kleinbuchstaben.

Die Rückruffunktionen `wcb::checkEntryFor*` prüfen, ob der Text im Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` nach dem Einfügen der Zeichenkette `str` vor dem Zeichen, das der Index `idx` bezeichnet, der Anfang einer ganzen Zahl, einer vorzeichenlosen ganzen Zahl nicht größer als `max`, einer reellen Zahl bzw. einer reellen Zahl im Festkommaformat mit höchstens `cnt` Stellen nach dem Dezimalpunkt wäre. Ist das nicht der Fall, brechen sie die Operation `insert` ab. `max` und `cnt` sollen nichtnegative Zahlen oder `*` sein. `max = *` bedeutet: keine obere Schranke für den Wert des Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widgets, während `cnt = *` für eine unbegrenzte Anzahl von Stellen nach dem Dezimalpunkt steht.

Die Rückruffunktion `wcb::checkEntryLen` prüft, ob die Länge des Textes im Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` nach dem Einfügen der Zeichenkette `str` größer als `len` wäre. Ist das der Fall, bricht sie die Operation `insert` ab.

Diese Rückruffunktionen sind in der Datei `wcbEntry.tcl` im Verzeichnis `scripts` implementiert. Sie geben eine leere Zeichenkette zurück.

SCHLÜSSELWÖRTER

callback, insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

before-`insert`-Rückruffunktionen für text- und ctext-Widgets

NAME

`wcb::checkStrsFor*`, `wcb::convStrsTo*` – before-`insert`-Rückruffunktionen für text- und ctext-Widgets

ÜBERSICHT

```
wcb::checkStrsForRegExp exp w idx args

wcb::checkStrsForAlpha      w idx args
wcb::checkStrsForNum        w idx args
wcb::checkStrsForAlnum      w idx args

wcb::convStrsToUpper        w idx args
wcb::convStrsToLower        w idx args
```

BESCHREIBUNG

Die Rückruffunktion `wcb::checkStrsForRegExp` prüft, ob der reguläre Ausdruck `exp` die Zeichenketten trifft, die in das text- oder ctext-Widget `w` eingefügt werden sollen und in der Liste `args` der Form `string ?tagList string tagList ...?` stehen. Trifft er sie nicht, bricht sie die Operation `insert` ab.

Die drei anderen Rückruffunktionen `wcb::checkStrsFor*` prüfen, ob die Zeichenketten, die in das text- oder ctext-Widget `w` eingefügt werden sollen und in der Liste `args` der Form `string ?tagList string tagList ...?` stehen, alphabetisch, numerisch bzw. alphanumerisch sind. Sind sie es nicht, brechen sie die Operation `insert` ab. Diese Prozeduren rufen nur die Rückruffunktion `wcb::checkStrsForRegExp` auf und übergeben ihr die Unicode-Muster `{^[[:alpha:]\n]*$}`, `{^[[:digit:]\n]*$}` und `{^[[:alnum:]\n]*$}`.

Die Rückruffunktionen `wcb::convStrsTo*` ersetzen die Zeichenketten, die in das text- oder ctext-Widget `w` eingefügt werden sollen und in der Liste `args` der Form `string ?tagList string tagList ...?` stehen, durch ihre Entsprechungen in Groß- bzw. Kleinbuchstaben.

Diese Rückruffunktionen sind in der Datei `wcbText.tcl` im Verzeichnis `scripts` implementiert. Sie geben eine leere Zeichenkette zurück.

SCHLÜSSELWÖRTER

callback, insert, text, ctext, widget

Lizenz

Deutsche Übersetzung © 2026 Gregor Ebbing.

Die Seiten wurden aus den Originalen nach Markdown umgewandelt und ins Deutsche übersetzt; Auszeichnung ohne Platz im Format (Verweise ohne Ziel, Layoutbilder) kann dabei wegfallen. Das sind die Änderungen; für Teile unter der Mozilla Public License 1.1 (tDOM) ist es die Beschreibung der Änderungen nach Abschnitt 3.3.

Die Übersetzung steht unter denselben Bedingungen wie das jeweilige englische Original. Diese Bedingungen folgen. Die Copyright-Angaben der Originalseiten stehen bei jeder Seite.

Multi-entry widget package Mentry, version 4.5.1 Copyright (c) 1999-2026 Csaba Nemethi (E-mail: csaba.nemethi@t-online.de)

This library is free software; you can use, modify, and redistribute it for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions.

This software is distributed WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Stichwortverzeichnis

I

index 3

M

mentry 3
mentryDateTime 30
mentryFixedPoint 35
mentryIPAddr 37
mentryIPv6Addr 38
mentryThemes 41
mentryWidget 44

W

wcbRef 55